

Machine Learning for ECB Inspection Reporting: A BERT Approach with Hierarchical Multi-Label Classification

CA' FOSCARI UNIVERSITY OF VENICE
Department of Environmental Sciences, Informatics and Statistics



Computer Science Bachelor's Thesis
Year 2023-2024

Graduand André Ramolivaz
Supervisor Claudio Lucchese

To my family...

Acknowledgments

Many thanks are due for this thesis. As a matter of facts this job represents for me the end of three fantastic academic years, both from an academic point of view and in terms of experiences. In these years, thanks to Ca' Foscari, I had the opportunity to meet extremely knowledgeable people from which I learnt really a lot. First of all, I must thank Professor Lucchese for his commitment and passion that shines through in his lessons and his research on interpretable models. Despite his various commitments, he immediately showed himself available to support me by providing me with useful advice and materials to improve my first real work experience until the end of writing of this work. Together with the professor, I also want to express my gratitude to Hendrik Brakemeier, who was my supervisor during my experience at European Central Bank (ECB) and certainly had an impact on this thesis. He shared with me extremely valuable lessons.

To that I would also add, that Ca' Foscari not only gave me value from an academic point of view but also from an humanity perspective. In addition to the professors, a thanks go to all my classmates who I have met in these three years. Each of you, a little in their own way, left a mark on me which certainly contributed to my education.

A final thank you goes to my family. Thanks to my sister Stephanie, who has always been a point of reference for me and who has been by my side not only to celebrate my successes but also during the bad moments. Furthermore, I am grateful to my father, Giuseppe, and my mother, Tiziana, for their sacrifices and continuous support. This relatively small achievement, together with the other experiences made in these three years with PricewaterhouseCoopers (PwC) in Milan, ECB in Frankfurt, and United Nations (UN) in New York City, will be just the beginning, I promise you.

Abstract

Machine learning (ML) applications in the financial and banking environment are becoming more common and crucial for improving efficiency. As financial institutions and regulators must adapt, the European Central Bank (ECB) is at the forefront of integrating these technologies to enhance its supervisory processes. This thesis presents a project developed during an internship in the ECB's Supervisory Technologies and Data Division, focusing on how ML, specifically Natural Language Processing (NLP), can be applied to improve the drafting and consistency of Assessment Reports (ARs) during inspections. More in details, the case study used, called MEDUSA (Machine Enhanced Drafting and User Support Application), show how NLP techniques like Hierarchical Multi-Label Classification and BERT, can be implemented in this context and can improve efficiency by standardizing and streamlining the creation of these reports.

It is in this regard that the thesis points out that ML can improve efficiency in financial operations, and highlights some of the important challenges for the interpretability and uncertainty of decision-making processes relevant in a regulatory and economic context.

Keywords Machine Learning, Hierarchical Multi-Label Classification, Natural Language Processing, Hugging Face Pipeline, BERT, Banking Supervision.

Contents

1	Introduction	1
1.1	Context and Background	1
1.2	Problem Statement and Objectives	2
1.3	Thesis Structure	3
2	Machine Learning in Reporting	5
2.1	Overview	5
2.1.1	Strengths and Limitations	8
2.2	State-of-the-Art	8
2.2.1	Machine Learning Techniques	8
2.2.2	Text Mining and NLP	11
2.2.3	Integrating NLP in Financial Reporting	14
2.3	Key Takeaway	14
3	MEDUSA	16
3.1	Project Overview	16
3.1.1	Objectives	17
3.1.2	Methodology	19
3.2	Learning Tasks	20
3.2.1	Multi-Label Hierarchical Approach	23
3.2.2	Formal Definition	24
3.2.3	Loss Functions and Metrics	25
3.2.4	Finding Similarity	28
3.2.5	Severity Prediction as Regression Learning Task	29
3.2.6	Handling of Long Findings	31
3.3	A Custom Huggingface Pipeline	32
3.3.1	The Huggingface Library	32
3.3.2	MEDUSA Pipeline	33
3.3.3	Data pre-processing	35
3.3.4	Fine-Tuning	36
3.3.5	TensorFlow vs. PyTorch	45
3.4	Key Takeaway	46
4	Impact Assessment and Prospects	48
4.1	Evaluation Methodology and Performance Analysis	48
4.2	Further possible Applications	51
4.2.1	LambdaMART for Interpretable Valuation	51
4.3	Key Takeaway	54
5	Conclusion	55

Chapter 1

Introduction

This chapter inaugurates the thesis by focusing on the context, the objectives, and the structure.

To begin with an insightful perspective, an important economist once said: [11]:

“Risk comes from not knowing what you’re doing” – Warren Buffett

Although Buffett was talking about investments, this can well apply to Machine Learning in banking supervision. The challenge isn’t just about developing and using advanced technology but also about mitigating risks through a deep understanding and evaluation of the tools we employ. This idea has been the guiding principle throughout the entire writing and research process of this work.

1.1 Context and Background

Recently, the science of Machine Learning and Artificial Intelligence (AI) has undergone an outstanding rise in applications within several industries, not excluding finance.

As estimated by McKinsey & Company [34], adoption of AI and ML in the financial services has the potential to generate anywhere from \$200 billions to \$340 billions in new annual value. In light of such estimates, the use of ML within the financial institutions, including the regulatory bodies, has become quite important. We center our attention on the European Central Bank’s use in the investigation for this study.

The European Central Bank is undoubtedly amongst the most powerful supervisory institutions aimed at making as well as implementing monetary policy to ensure it is stable for the Eurozone. Some of its core responsibilities include the determination of interest rates ensuring financial stability and proper running of the banking process in the member countries. It would, therefore, require the ECB to rely heavily on technologies with respect to data management, analysis, supervision, and making informed decisions at this level of complexity and volume of transactions. Here is where ML comes into play. Machine Learning is that part of Artificial Intelligence concerned with the creation of algorithms and statistical models that allow computers to acquire experience of the performance of certain tasks. [19, 28, 29] ML is usually applied to process huge data volumes, creation of

patterns, and making predictions.

ML integrated into the financial sector is not just a technological advancement, but a crucial part supporting precision and efficiency. In fact, the ECB is leveraging more on ML also on its duty related to banking supervision. As nearly all well-structured major industries nowadays, dedicated divisions *Supervisory Technology and Data Division* and the *Technology and Innovation Division* have been created in the Single Supervision Mechanisms (SSM). [16, 3]

1.2 Problem Statement and Objectives

Despite obvious advantages and the rapid diffusion of ML technologies, there are still important challenges and limitations that must be considered, particularly within a regulatory framework like the one ruling the ECB. The most important problem that this thesis addresses is the development of powerful technologies that are not free from all their complexities. These technologies should also be reliable, interpretable, and able to capture uncertainties effectively.

Interpretability of models is one of the most important challenges to the implementation of ML within banking supervision. Unlike in the case of classical statistical techniques, many ML algorithms are very often “black box” wrapped, and the user has little notion of how it makes a decision. In Fig 1.1 we show the difference between Interpretable and Black Box models, although the output may be the same in the Interpretable models we are able to justify why while in the other one nothing is known except for the input and output. In most cases, this usually poses problems for regulatory purposes, whereby clear and understandable decision-making processes are paramount. For example, if an ML model flags a transaction as suspicious, then the supervisors must understand why to take proper action and ensure compliance with all legal standards.

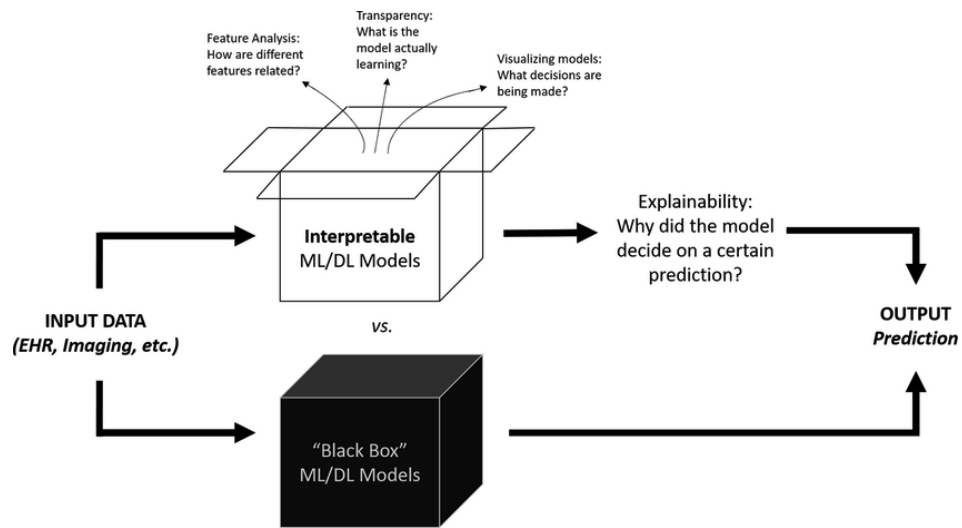


Figure 1.1: Black Box and Interpretable Models

Another critical challenge is how to manage the inherent uncertainties. In most cases, financial data can be noisy and incomplete, hence likely to result in uncertain or unreliable predictions. This then creates significant risks when making

decisions that could have an effect on compliance with regulations. For example, an ML model predicting loan defaults might work very well on historical data, missing unforeseen economic shifts and making wrong predictions that might cause instability.

Therefore, the objectives of this thesis are twofold:

- **Assess the implementation and impact of ML:** By discussing and showing some parts of the developed application of a ML project, demonstrate its complexity including learning tasks, and assess the impact of ML algorithms on ECB banking supervision operations with this case study.
- **Identify solutions for interpretability and uncertainty** Highlight ethical concerns and responsibility-related limits when addressing interpretability and uncertainty issues.

Ethics in ML range from issues of data privacy and security to algorithm bias and potential discrimination.[4, 2] Also, clear lines of responsibility should be drawn for decisions made by ML systems. Especially in a regulatory context, it becomes very relevant to clarify who is responsible for the actions taken in consequence of the predictions made by ML—who developed the technology, financial institutions that use it, or the supervisory authorities that oversee its implementation.[1]

1.3 Thesis Structure

The thesis is structured into five chapters to address the expressed objectives of the problem statement by using as a case study a machine learning project called MEDUSA. This tool is now used for enhancing drafting and consistency in some of the ECB supervisory reports.

Following the introduction, the second chapter covers the level of development of the project-related state of the art. The third chapter introduces the case study, describes the objectives, methodology, and key learning tasks of the project. The fourth chapter makes an assessment of the impact of the solution developed and its performance; it also checks for other applications like the use of LambdaMART for interpretable asset valuation. The conclusion summarizes the key findings and the ethical and responsibility-related limits.

Summarizing and reiterating the results of this thesis, this work is the result of an internship at the ECB. During this experience, I had the opportunity to see how ML was strongly present and fundamental to facilitate the tasks even if full of complexities, both related to development and ethics. With this work, I want to summarize and synthesize the main critical points. To do this I use a project that I believe was the most interesting and from which more points of reflection could have been extracted. The project in question was already well underway at the time of the beginning of my internship and almost in the deployment phase so many phases of the development (i.e. model training and main structural development choices) presented were not developed by me in first person. Anyway I still had the chance to study them and assist in the development of new features. The project in question demonstrates the application of a BERT-based Hierarchical

Multi-Label Classification on financial reporting, specifically in drafting Assessment Reports showing a general overview of the project, specific learning tasks needed, and main sketch of Python used to build MEDUSA.

MEDUSA, built upon Hugging Face pipelines, categorizes findings useful for the build of the reports and predicts them. Additionally, I provide possible future developments by exploring other potential applications in MEDUSA, such as the use of LambdaMART for interpretable valuation, highlighting its usefulness in enhancing decision-making transparency. Throughout, the research also emphasizes the literature behind ethical challenges and responsibility-related limitations in deploying machine learning models in regulatory contexts, such as ensuring model interpretability, managing uncertainties, and delineating accountability.

Chapter 2

Machine Learning in Reporting

This chapter aims at a state-of-the-art review related to this thesis. It proposes a review of tools and algorithms at the core of the context established in the introduction and on the project to be discussed later.

2.1 Overview

Automated reporting tools have become essential in banking supervision. These tools usually merge various technologies together to process vast amounts of data, identify patterns, and generate reports that help in decision-making processes.

In this regard, reporting is supposed to involve the structured collection, processing, and summarization of data into formats that present some insight to the decision makers by which they could make some positive decisions [21]. Automated reporting helps ensure that the right information will become available in good time for regulatory compliance and key, data-driven decisions to be made within even the tightest deadlines that finance and banking necessitate [40, 23, 35].

A number of public and open source automated reporting tools have been developed to encounter the challenges. Those tools range from complete platforms to specialized applications focusing only on specific aspects of report analysis. Some of them are drawn below.

Apache Kafka (Fig. 2.1) Apache Kafka is a stream-processing platform primarily used for building real-time data pipelines and streaming applications [22]. It allows to process and analyze data streams, supports high-throughput, and scalable processing, making it a valuable tool also for handling the large volumes of financial data. For example JPMorgan Chase uses Apache Kafka to provide real-time operational reporting by integrating data from its subsidiaries and corporate platforms. In this scenario, Kafka continuously collects updates on all those internal processes-from every transaction and day-to-day operation-collecting data on one ongoing stream. Thus, the bank can periodically generate updated, detailed reports of operational performance so that comprehensive, immediate insight into the effectiveness of business processes is attained. In real-time, aggregation of this information enables the manager to identify operational delays, bottlenecks, or other problems that may be occurring and act upon them with haste. [13, 22]



Figure 2.1: Apache Kafka Logo

KNIME (Konstanz Information Miner) (Fig. 2.2) KNIME is an integrated platform for data analytics, reporting, and integration.[10] This tool offers a user-friendly interface for the creation of data workflows, integration from different sources, and execution of many analytics. KNIME thus also enables machine learning and data mining and might be applied to automate even reporting tasks. An interesting case of using KNIME for optimizing financial reporting is the project realized by Day5 Analytics for an important client in the sector of financial management. More than 500 financial processes were pursued for automation, and this objective was pursued in order to increase operational efficiency and reduce human errors. For instance, one project automated an accruals management process, which was done by hand in Excel, taking roughly 2.5 days each month. Using KNIME, this was centralized and taken down to less than half a day. This saves roughly five weeks of work per year. [26]



Figure 2.2: KNIME Logo

RapidMiner (Fig. 2.3) RapidMiner is a platform for specific for data science and machine learning [20]. It offers a range of tools for data preparation, modeling, and deployment. RapidMiner also have a visual workflow designer and extensive library of pre-built models enabling easier implementation. One interesting example of using RapidMiner for easing reporting by gaining insight into operational improvements within the financial industry is by Nasdaq, the famous electronic stock exchange. Nasdaq said it has integrated RapidMiner's data mining capabilities into its IR Insight software in order to help customers analyze massive data sets in real time.

Mainly, Nasdaq was trying to let customers pool data from a number of sources, including license data and manually entered information, and unlock fresh value from that data. Using RapidMiner, Nasdaq enhanced its capability in visualizing and interpreting large volumes of data within the shortest time and helped customers create reports very well clarified, useful for internal decisions over business. [14, 8]



Figure 2.3: RapidMiner Logo

TensorFlow (Fig. 2.4) One of the most popular machine learning frameworks, developed by Google, is TensorFlow [6]. In the context of financial reporting also TensorFlow facilitates the creation of sophisticated models for data analysis and prediction.



Figure 2.4: TensorFlow Logo

NLTK (Natural Language Toolkit) NLTK is a comprehensive Python library for natural language processing. It carries out some of the fundamental operations in processing, which includes text processing, classification, tokenization, stemming, and many others. Several institutions leverage NLTK to analyze textual data like transaction records and compliance documents to obtain meaningful insights and automate the generation of detailed reports. A real use case involves the use of NLTK in transactional analysis and compliance. Financial institutions engage NLTK in the processing of big data sets relative to transaction records and compliance documents for information extraction from them, finding patterns indicative of breaches in compliance or suspected activities. This automation cuts down manual labor by a great degree and enhances the accuracy of reporting. [9]

Hugging Face (Fig. 2.5) Finally, Hugging Face provides a library that gives a wide range of models and tools in NLP, therefore easing the application of machine learning techniques to text processing tasks. This includes pre-trained models for tasks such as text classification, named entity recognition, and sentiment analysis. His pipeline is ease of use and learn. [18] The case study proposed in the following thesis will be focusing on the Hugging Face library.



Figure 2.5: Hugging Face Logo

2.1.1 Strengths and Limitations

Each of these tools brings unique strengths. However, they also present certain limitations that need to be addressed.

Apache Kafka and TensorFlow, for example, are very scalable and perform well with large data volumes and real-time data processing. High scalability and performance are required in streams typical in financial transactions. Other platforms, such as KNIME and RapidMiner, provide flexible frameworks supporting the broadest range of data sources and methods of analytics. This will greatly enhance their applicability within different tasks. Moreover they are capable of performing predictive analytics by using machine learning capabilities available through tools such as TensorFlow and RapidMiner. It makes full NLP functionality with NLTK and Hugging Face available for conducting text analysis, which is very important in dealing with unstructured data from financial documents.

Advanced functionalities almost always come with a steep learning curve, often requiring special knowledge to implement and maintain. The integration of these tools with pre-existing systems and data sources also poses problems.

Keeping both the strengths and limitations in mind, this thesis proposes a pipeline using Hugging Face for ease of use, versatility, and NLP capabilities. With their pre-trained models, Hugging Face facilitates the implementation of state-of-the-art machine learning techniques; thus, it is well suited to develop a robust reporting tool.

2.2 State-of-the-Art

By understanding the state-of-the-art of machine learning techniques and text mining algorithm, even if not really necessary for the examination of the case study, we can appreciate more their applications and the reasons for selecting specific methods in some steps.

2.2.1 Machine Learning Techniques

Notably, the main macro machine learning techniques applied within the framework of financial reporting and supervision are supervised learning, unsupervised learning, and reinforcement learning.

Supervised Learning In supervised learning, a model is trained using a labeled dataset in which the input-output pairs are well known [19]. This technique has been applied in solving a number of problems involving classification and regression. On the side of classification, the model should learn to group new instances into predefined classes from historical data. For example, a model using supervised learning can be trained to classify a transaction either as fraudulent or nonfraudulent based on the learning derived from past labeled. [38, 37] In Fig 2.6 there is a graphic example in which the model trained, the line, can classify each entry into red or blue according to the features given in input.

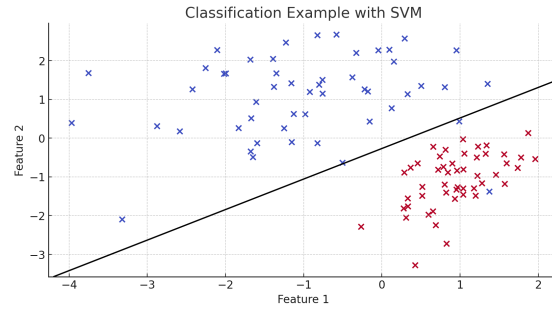


Figure 2.6: Classification Example

In regression tasks, the model predicts continuous outcomes. For example, such supervised learning can be used to predict stock prices or credit scores from historical data. A learning procedure minimizes the difference between the model's prediction and the true values, usually using some loss function like the mean squared error. In Fig 2.7 there is a graphic example in which the model trained, the line, can predict the numerical target value based on the feature given in input.

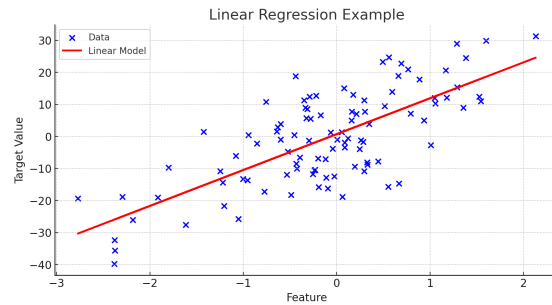


Figure 2.7: Regression Example

Popular algorithms for supervised learning include:

- Linear Regression: Used for predicting continuous values.
- Logistic Regression: Used for binary classification.
- Support Vector Machines (SVMs): Used for classification tasks.
- Neural Networks: Used for complex pattern recognition tasks.

Unsupervised Learning In contrast, unsupervised learning deals with the identification of patterns in data that has no labeled outcomes. The method is of key importance in clustering and dimensionality reduction. Unlike supervised learning, unsupervised learning seeks to guess an outcome variable; it discovers the underlying structure of the data itself.

Clustering unsupervised algorithms group data points together [38, 37]. In financial supervision, it can be used to spot anomalies or even suspicious activities by way of detecting transactions way far from the rest or patterns established earlier. For instance, transactions that cluster together could indicate a common fraudulent behavior not known before.

Another application of unsupervised learning is Dimensionality Reduction. Dimensionality reduction techniques such as PCA reduce the complexity of datasets by bringing down the number of variables in consideration. This becomes very useful during the analysis of financial data, where one has to deal with a lot of variables that sometimes obscure the underlying trends. They help in visualization and improve the performance of many other machine learning models. Popular algorithms for unsupervised learning include:

- K-Means Clustering: Groups data into k clusters based on similarity.
- Hierarchical Clustering: Creates a tree of clusters to represent data hierarchy.
- Principal Component Analysis (PCA): Reduces the dimensionality of data.
- Neural Networks: Used for complex pattern recognition tasks.

Reinforcement Learning Reinforcement learning (RL) is based on the idea of agents learning to make decisions by receiving rewards or penalties.[36] In such a strategy, the agent learns how to interact with an environment in taking actions so as to maximize the cumulative rewards. RL is good compared to supervised and unsupervised learning in dynamic and sequential decision-making processes. It can also be used in finance to optimize trading strategies. For example, train an RL agent when to buy or sell some stock according to the conditions of the market observed so that it maximizes the total return over time. The agent learns from its actions through trial and error methods and gets feedback in the form of rewards or penalties for its actions.

Components of reinforcement learning include the following:

- State: The current situation of the agent.
- Action: The decision or move made by the agent.
- Reward: The feedback received after taking an action.
- Policy: The strategy employed by the agent to decide actions.
- Value Function: Estimates the expected reward for a given state.

These are both fundamental and basics behind the development of machine learning models. Hugging Face pipeline leverages these tenets through its pre-trained models and fine-tune models for a specific task, thereby reducing complexity and required resources.

Additional notes on Neural Networks and Deep Learning Neural networks are partly inspired by the structure and function of the human brain, comprising a connected network of multiple layers of nodes called neurons (Fig. 2.9).

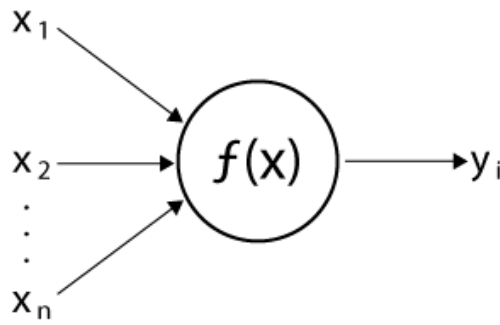


Figure 2.8: Artificial Neuron [27]

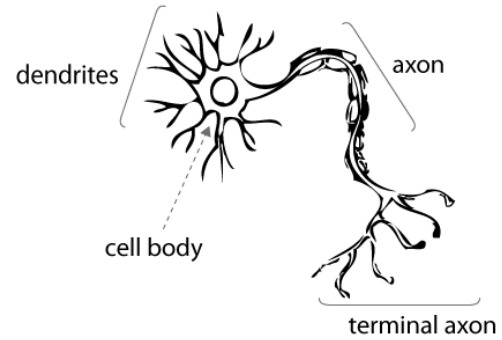


Figure 2.9: Neuron [27]

Each Artificial Neuron, as Fig. 2.8 shows, receives inputs, processes them using a weighted sum, and produces an output. These connections, or weights, determine how much influence one neuron has on another. [27] During training, these weights are adjusted to minimize errors in predictions. To introduce non-linearity, neurons use activation functions like sigmoid, ReLU (Rectified Linear Unit), and tanh. This non-linearity enables neural networks to learn complex patterns in data.

Neural networks (Fig. 2.10) are made up of an input layer that receives data, hidden layers that process it, and finally an output layer that comes up with predictions. In training, backpropagation adjusts weights based on the gradient calculation for a loss function—a measure of difference between predicted and true values. Common optimization algorithms like gradient descent and Adam help minimize this loss.

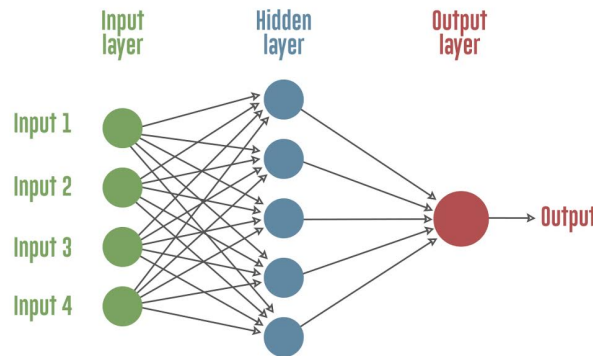


Figure 2.10: Neural Networks Layers [27]

Deep learning involves neural networks with many layers, known as deep neural networks. These networks automatically learn features from raw data, reducing the need for manual feature engineering. Deep learning is particularly effective for tasks such as image and speech recognition, and our NLP scenario of application in financial reporting.

2.2.2 Text Mining and NLP

Going back to the scope of our project is important to remember that the focus will be on assist in drafting ARs. For this reason is also necessary to explain further and give an overview of NLP and Text Mining. Text mining and NLP are

sub-classes of ML that can be used to provide an analysis on data from reports, not limited to that. NLP is a variety of processing and understanding human language techniques that enable extracting meaningful information from the text data.

The choice of tokens as the unit of measurement indicates the level of precision that is called for, since tokens are minimal units in a textual analysis. Consider a sentence: "Dr. Smith, a renowned scientist, won the Nobel Prize in Physics in 2020." If we measure the text by words, we count

Words

```
"Dr.", "Smith,", "a", "renowned", "scientist,", "won", "the",  
"Nobel", "Prize", "in", "Physics", "in", "2020."
```

This method may not consider punctuation or special characters. In its place, character measure would include each and every letter, space, and punctuation, hence being inconsistent due to these formatting differences and variation in punctuation of the text.

Characters

```
"D", "r", ".", " ", "S", "m", "i", "t", "h", ",", " ", "a", " ",  
"r", "e", "n", "o", "w", "n", "e", "d", " ", " ", "s", "c", "i", "e",  
"n", "t", "i", "s", "t", ",", " ", "w", "o", "n", " ", "t", "h",  
"e", " ", "N", "o", "b", "e", "l", " ", "P", "r", "i", "z", "e", " ",  
"i", "n", " ", "P", "h", "y", "s", "i", "c", "s", " ", "i", "n",  
" ", "2", "0", "2", "0", "."
```

Finally if we tokenize the text, we break it into meaningful units:

Tokens

```
["Dr.", "Smith", ",", "a", "renowned", "scientist",  
",", "won", "the", "Nobel", "Prize", "in", "Physics",  
"in", "2020", "."]
```

It puts more correctly the punctuation and special characters as separate tokens, therefore providing a more complete representation of the text. This way, using tokens instead of words or even characters, the length of text ranges is better expressed and coherent, thus able to accommodate variability and complexity of different sections within the reports.

Another important task in NLP and Text Mining is Named Entity Recognition. It consists of the identification and classification of the entities of a text into predefined categories, like names of a person, organization, location, monetary value, and so on. In this way, through NER, relevant information could be obtained from the financial documents for consideration in the reporting process. Lastly, another major task assigned to NLP and that will also be discussed in this paper will be that of text classification. That is, assigning predefined categories to text documents, as explained more in detail in Chapter 3.

Transformer Models

The Transformer models represent a particular type of Neural networks that shook the world of NLP, became central to many modern applications used in financial supervision, and are used, of course, in the one that will be described later. These models, such as BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer), are sequential models by construction for processing sequential data and learning context based on the relative relations of words in a sentence, independent of their position. This is made possible through self-attention mechanisms that let the model weigh the importance of each word in relation to others.

Besides all the advantages that a transformer model has, it increased language understanding and improved accuracy for most tasks, specifically in machine translation and sentiment analysis. Additionally, their architecture makes it much easier to parallelize the processing of data, which makes them faster and more scalable. Thanks to this no model has to be created from scratch; they have an excellent starting point, and they save a lot of time and resources in the development of NLP applications.

BERT (Bidirectional Encoder Representations from Transformers): BERT was developed by Google. It is designed to understand the context of a word in both backward and forward directions. As a result, it is very effective for any language-intensive tasks such as text classification, NER, and question answering. That simply means BERT considers all the words preceding and following simultaneously, looking at a full context for a word. For example in “*The bank will not approve the loan*” the word “*bank*” could mean a financial institution or the side of a river. BERT’s bidirectional approach allows it to understand that “*approve the loan*” indicates that “*bank*” refers to a financial institution [15].

The two major steps in the training process of BERT are pre-training and fine-tuning [32]. During pre-training, a huge amount of text data, like the content of the whole of Wikipedia and books, is trained on BERT [17]. It contains two unsupervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). In MLM, some of the words in a sentence are randomly masked, and the model learns to predict those masked words contextually. For instance, in the sentence “*The [MASK] is on the table*” BERT must predict that the masked word is “*book*.” In NSP, BERT is trained to understand the relationship between sentences by predicting if a second sentence logically follows the first one. For example, given the pair of sentences “*She opened the door*” and “*It was raining outside*” BERT learns to predict whether the second sentence is a logical continuation of the first.

After pre-training, BERT is fine-tuned using labeled data for specific tasks. The pre-trained model is fine-tuned for particular applications: text classification, NER, or question answering. For instance, if BERT has to be fine-tuned in classifying a movie review as positive or negative, it will be trained on a dataset where each review is annotated with its sentiment. Through such elaborate pre-training, this leaves BERT in a position where it can perform zero-shot on these tasks, needing at most small quantities of fine-tuning data for accuracy.

GPT (Generative Pre-trained Transformer): On the other hand, GPT (Generative Pre-trained Transformer) models are majorly generative models targeted at text generation and language understanding by means of next-word prediction in a sequence. This makes them quite useful during summarization, translation, and question-answering tasks. Unlike BERT, GPT models are unidirectional, typically processing text left to right, meaning they predict the next word from the previous words in a sentence. However, GPT models are less fitting for contextual understanding, aspects for which BERT is very good. Therefore, in our case study, BERT would be more relevant than GPT due to its unparalleled context comprehension ability [39, 31].

Hugging Face Transformers: Again the Hugging Face Library comes to our need by providing easy access to various transformer models, including both BERT and GPT [41, 17].

2.2.3 Integrating NLP in Financial Reporting

Despite all these advances, there are still a number of challenges that must be overcome with the application of NLP to financial reporting. The texts are mostly domain-specific, full of jargon, and difficult to understand even for general NLP models. Moreover, financial documents are generally very long, and we will see soon that most current NLP models have many limitations in processing long texts.

While many works have been carried out on improving model performance within NLP, little research has been done on the specific challenges of financial texts. On the other hand, while such engines as BERT and Hugging Face Library bring improvements, the literature normally marks their limitations in specific domains, such as finance. For instance, the interpretability of transformer models is still a critical issue, as recently claimed by studies[33]. Also, it is not easy to incorporate such models with a running financial system as it entails a good amount of system compatibility and data privacy issues to be taken care of.

2.3 Key Takeaway

In this chapter, we considered the integration of ML into financial reporting, paying great attention to the role of automated reporting tools. This chapter underlined both the strengths of such tools—with high scalability, versatile frameworks, and comprehensive NLP functionalities though also their limitations in terms of steep learning curves and integration challenges.

We also focused on machine learning techniques:

- **Supervised Learning:** Trains models on labeled datasets to make classification and regression predictions.
- **Unsupervised Learning:** Identifies patterns lacking labeled outcomes that are useful for tasks in clustering and dimensionality reduction.
- **Reinforcement Learning:** Basically, agents learn how to make decisions through rewards or penalties. This is very suitable for problems of dynamic decision-making.

Finally, this chapter touches on text mining and NLP with a focus on tokenization, NER, and text classification.

Chapter 3

MEDUSA

This chapter explores the project used as a case study to explain a ML solution on ECB operations. The project, known as MEDUSA as already mentioned aims to revolutionize how ECB supervisors manage and analyze draft Assessment Reports (ARs). In this context, ARs are the detailed documents that supervisors at the ECB create to assess financial institutions under their supervision and their activities with regard to compliance. MEDUSA, which stands for **M**achine **E**nhanced **D**rafting and **U**ser **S**upport **A**pplication, enhances this drafting phase with ML techniques that guarantee the coherence of the ARs.

First of all, we will introduce the project in this chapter. We will do an initial contextual and problematic analysis of the project; later, we focus on technical aspects: learning tasks of the tool and application of the Hugging Face pipeline.

3.1 Project Overview

Preparing ARs conventionally necessitates much manual effort, namely, the time-consuming searching, reading, and consulting of many domain experts. Not only is the process time-consuming, but it also has inconsistencies—variations in interpretation and subjective integration of internal models, regulations, and guidance are usual. An AR will typically need accurate legal references, appropriate wording, clear topic labeling, and proper scope and severity assessment of issues for its credibility and usefulness.

MEDUSA will concisely disrupt the entire preparation process with the aid of sophisticated machine learning techniques. Based on applied NLP and hierarchical multi-label classification, MEDUSA will support the end user with standardized legal references and wording suggestions for compliance with the relevant legal and regulatory framework, useful labels like topics, and expected references, support for assessing scope appropriateness, and plausibility of severity of the identified issues. Moreover, MEDUSA provides links to regulations and similar findings, which helps one reference and compare relevant information.

System Integration: Since MEDUSA is ultimately a ML-based tool, and given the initial definition of ML, its development necessitates leveraging on “*experience*”. To achieve this, after an initial training phase, the tool is also seamlessly integrated with other ECB storing data systems such as DARWIN (Data Analysis

and Reporting Validation Interface).

This integration allows MEDUSA to access a comprehensive and continuously updated database of historical reports, enriched with new data from ongoing missions. This integration is crucial to the “*experience*” for several reasons:

- **Historical Insights:** In this way the tool is able to taps into a vast repository of historical data that can be used to understand past decisions and supervisory trends.
- **Real-time Updates:** The integration also ensures that real-time updates from ongoing supervisory activities are received. This feature provide up-to-date support and insights on current cases. It also means that any changes in regulatory guidelines or internal model guidance are quickly reflected in operations.
- **Uniformity:** Finally it allows for consistent methodologies and interpretations across different teams and missions by ensuring that all stakeholders have access to the same data and analytical tools.

3.1.1 Objectives

Building upon the initial overview and context, we now go into core functionalities and strategic objectives. MEDUSA has several key features:

- **Search Function:** This feature allows a user to search the database of reports and obtain relevant historic findings and evaluations for review. Supervisors can, through the utilization of a robust library of historic reports, understand trends and historic decisions.
- **Classification of Issues:** MEDUSA does auto-classification of the identified issues against predefined criteria, which are set up with consideration for regulatory standards and expectations. Automated classification ensures each and every issue is properly labelled and organized for easier review and analysis.
- **Syntax and Suitability Checks:** MEDUSA checks all the references as regards their syntax and suitability for correctness and conformity with the relevant legal frameworks.
- **Detection of Similar Occurrences:** The tool picks up similar issues across various reports, thus enabling supervisors to recognize the trends that may point to systemic risks or areas for more critical review.

MEDUSA’s objectives can now be more evident, but for clarity, let’s state them:

1. **Enhancing Efficiency:** It reduces the time and effort that human supervisors put into the laborious parts of AR preparation, freeing them to undertake more complex analytics tasks and decision-making processes.
2. **Promoting Consistency:** The tool standardized the treatment of findings and issues across different reports and supervisors. Therein is the standardization that different reports are going to be of the same standard of quality and accuracy.

3. Improving Decision Quality: Equips supervisors with comprehensive tools to make more informed, data-driven decisions.

Let us take examples through possible real applications to further outline the strategic objectives and actual usage in day-to-day operations.

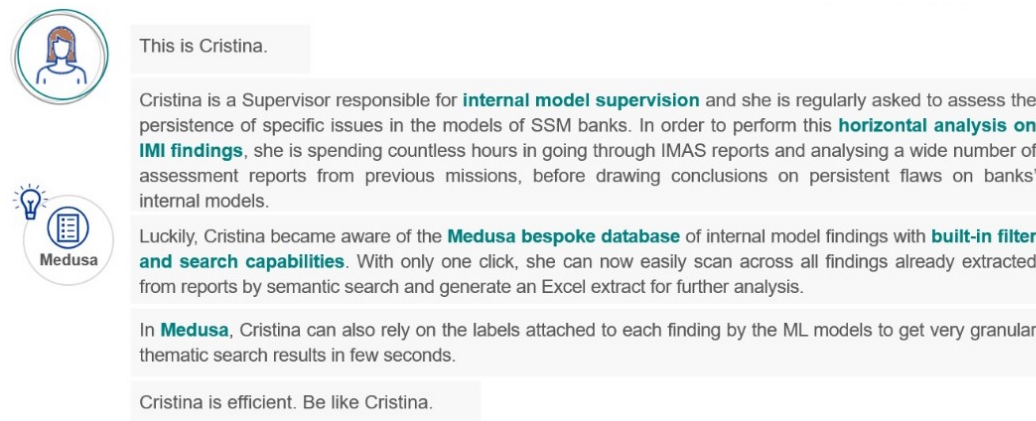


Figure 3.1: User Story 1: Horizontal analysis on IMI findings

User Story 1 (Fig.3.1): In the internal model oversight, Cristina used to go through numerous reports stored in IMAS (Information Management System)¹, spending long hours, to filter out some recollecting problems that may crop up in SSM banks. Horizontal analysis had been revolutionized with the advent of MEDUSA for her. Cristina could do rapid, granular, and refreshable views across multiple reports. This gives her the opportunity not to waste time handpicking the documents to read but, at the same, time increases the accuracy of her analysis through granular thematic searches. Plus, it also helps to develop quick Excel extracts within the system to take things for further analysis so that not a single important insight is left behind in any of those complex data sets.

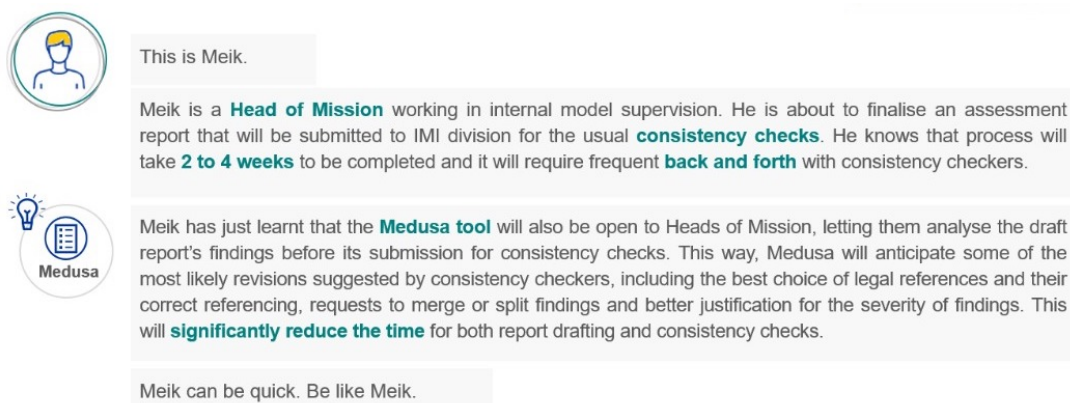


Figure 3.2: User Story 2: Draft of assessment reports

¹IMAS is the Management System where supervisors have direct, online access to information related to the supervised institutions, allowing them to share information with the ECB or an NCA.

User Story 2 (Fig.3.2): One of the challenges of the Heads of Missions in internal model supervision is the finalizing of assessment reports. Assessment reports are considered to be intensive in terms of consistency checks, hence long rounds of back-and-forth revisions. MEDUSA makes the tool available very early on to the Heads of Mission, offering them the opportunity to preempt most of the revisions and dramatically improve the drafting process. This kind of pre-emptive measure does not only speed up the completion of the report, it also enhances the quality and coherence of the ultimate submissions.

3.1.2 Methodology

The whole project can be split into a few macro sub-subject matters that shall act as chief pillars of the technological approach. They are as follows: Algorithm Development and Training, System Integration, User Interface Design, Validation and Testing, and Deployment. However, in this study, we will focus on the high level concerning the Algorithm Development and Training phase of the study, detailing some of the decisions made, which learning tasks were assigned to MEDUSA, and other relevant aspects.

Just for completeness, let us outline all the steps that are necessary in this phase:

Data Collection and Manipulation: This is the very first step where many model reports are evaluated with much scrutiny to generate information. Now, data can be divided into two categories: structured and unstructured data. The former refers to information which is organized and formatted in a manner so that it will be easily searchable and analyzable, usually stored in a database or spreadsheet with defined columns and rows. On the other hand, unstructured data is that which does not have any predefined format; much like our ARs. An important activity carried out during this phase is the adaptation of the extracted information into a format that would be efficiently processed by the next phases of the algorithm development.

Data Exploration, Wrangling, and Validation: The next step after gathering data would be to ensure that the data is in good quality and in usable form. It means it needs standardization and validation to clean the data. Tools such as regular expressions can also be applied to find specific patterns, say legal references in text documents. That would be a process for organizing data in structured form and enhancing it with relevant metadata for further analysis and modeling.

Data Modelling: In this stage, a data model would be trained on some requirements. This model is built for tasks such as named entity recognition or text classification.

Text Mining and Natural Language Processing (NLP): Text mining is a process for extracting information from text data. A basic framework is developed to identify and associate real-world observations against predefined categories. This mapping job is done using a tool called MEDUSA, that generates a vector of probabilities against various observations. It is a very basic step in making sense out of unstructured text data and drawing meaningful insights from it.

Supervised Learning: Supervised learning is the stage where the software is

trained to improve its accuracy in carrying out specified tasks, Named Entity Recognition in this case. This involves using labeled training data with tools like GoldParse in spaCy that provides all the annotations necessary for the training of such a model effectively. The goal is to get the model to do the recognition and sorting of data with more accuracy based on the training it gets.

Reinforcement Learning: One can optionally further improve the model using reinforcement learning. This is a strategy to enhance the predictive ability and accuracy of the model by using related datasets over time. The reinforcement learning gives the model the capability to optimally make decisions based on the feedback received in terms of performance which modifies its parameters.

Given the nature of the problem, MEDUSA uses a knowledge discovery process to identify patterns, extract insights, and convert the raw data from ARs into useful information. In consequence, a special focus on the methodology described above is placed on:

- **Data Selection:** Selecting appropriate data sets, including relevant ARs and other documents containing valuable information about financial institutions' activities and compliance.
- **Data Preparation:**
 - **Cleaning and Preprocessing:** Elimination of irrelevant or incorrect data entries; handling missing values to ensure accuracy and reliability.
 - **Transformation:** Transforming data into a suitable format for analysis, normalizing data, creating new derived attributes, and deleting irrelevant variables.
- **Strategy and Algorithm Selection:** Choosing the right data mining strategy and algorithms that align with the nature of data and project's goals.
- **Pattern Analysis:** Examine the patterns generated by the algorithms and interpret the results to infer actionable insights, making sure that the findings are relevant for improving drafting and consistency of ARs.
- **Iteration and Refinement:** It is a step-by-step refinement and improvement of the process in view of new insights and challenges that necessarily begin in a quest for continuous improvement and adaptation.

3.2 Learning Tasks

Moving toward the heart of MEDUSA, let's describe what the Learning Tasks is responsible for. Before doing that, let's reiterate that everything related to the tasks is focused on findings.

Finding 6	The mapping of trades to the portfolio hierarchy exhibits deficiencies
Legal Requirement	Articles 367(1)(a) and 367(2)(e) of the CRR
Criteria	According to Article 367(1)(a) of the CRR, the IMA shall capture accurately ...
Condition	The controls ensuring that full revaluation books and sensitivities are mapped to the portfolio hierarchy exhibit the following deficiencies ...
Root Cause	Books may fail to get mapped to the hierarchy for different reasons...
Effect	The lack of clear criteria for remediation of unmapped trades, the backlog of unmapped books and the non-consideration as part of the RNIV process may lead to...
Severity	F3

Figure 3.3: Example of finding

Findings, like the example in Fig.3.3, are well structured English texts, taken from comprehensive ARs. These findings could range between 100 and 3000 tokens in length and normally address matters of regulatory compliances, financial variations, or technical normality. There are four necessary structural components of a finding (Fig.3.2): **Legal Requirement**, **Criteria**, **Condition** and **Severity**. This will enhance clarity and standardization: each finding might, in turn, be decomposed into small blocks called *protofindings*. These *protofindings* use standardized wording and legal references to ensure consistency and precision. These are of prime importance in that they are essential in highlighting key issues during the drafting of reports. Let's see an example (Fig.3.5, Fig.3.6):

Finding 11	Shortcomings in the calculation of PD estimates	
Legal Requirement	Article 4(1)(78) of the CRR	Mandatory structural components of a finding Idea: Break down each finding into a set of smallest building blocks, the <u>protofindings</u>, with standardised wording and legal references.
Criteria	Article 4(1)(78) of the CRR in conjunction with Article 48 of the EBA CP on GL.	
Condition	In the computation of the default rate, the bank does not include credit risk offers for which no amount has been drawn at the observation moment and at any end-of-month during the following 12 months. The denominator does not include all off-balance sheet items and hence also credit risk offers without any drawn amount. In addition, the bank counts multiple defaults in the same year multiple times.	
Severity	F3	

Figure 3.4: Example: A finding from an AR

Finding 11	Shortcomings in the own estimate of PD	Assign this protofindings ' label:
Legal Requirement	Article 4(1)(78) of the CRR	Article 179(1)(a) of the CRR
Criteria	Article 4(1)(78) of the CRR in conjunction with Article 48 of the EBA CP on GL.	An institution's own estimate of PD shall incorporate all relevant data.
Condition	In the computation of the default rate, the bank does not include credit risk offers for which no amount has been drawn at the observation moment and at any end-of-month during the following 12 months. The denominator does not include all off-balance sheet items and hence also credit risk offers without any drawn amount. In addition, the bank counts multiple defaults in the same year multiple times.	The Supervised Entity did not incorporate all relevant data in its own estimate of PD.
Severity	F3	Observations: - Legal references used are different - The protofinding is not exhaustive on the issues

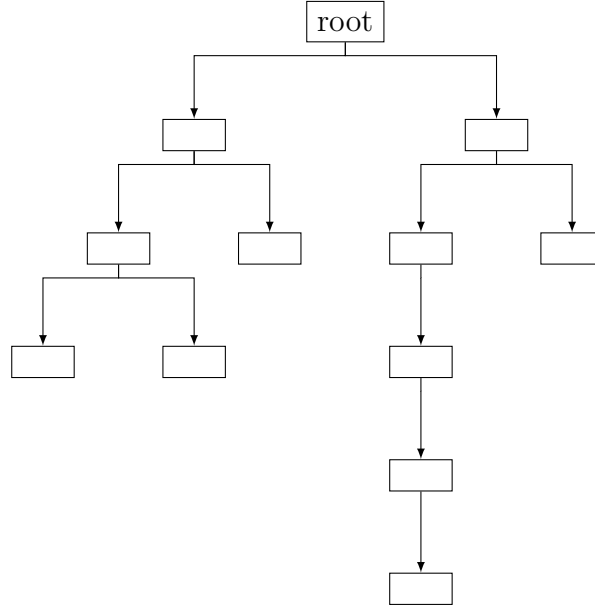
Figure 3.5: Example: Assigning a *protofinding* label

Finding 11	Shortcomings in the own estimate of PD	
Legal Requirement	Article 4(1)(78) of the CRR	Article 180(1)(a) of the CRR together with par. 77 of the EBA GL on PD and LGD
Criteria	Article 4(1)(78) of the CRR in conjunction with Article 48 of the EBA CP on GL.	Institutions shall estimate PDs by obligor grade from long run averages of one-year default rates. Par. 77 of the EBA GL on PD and LGD specifies that defaulted obligors should be counted only once in the default rate calculation, even where the obligor defaulted more than once during the relevant one-year period.
Condition	In the computation of the default rate, the bank does not include credit risk offers for which no amount has been drawn at the observation moment and at any end-of-month during the following 12 months. The denominator does not include all off-balance sheet items and hence also credit risk offers without any drawn amount. In addition, the bank counts multiple defaults in the same year multiple times.	The Supervised Entity counts multiple defaults in the same reference period multiple times.
Severity	F3	

Figure 3.6: Example: Assigning another *protofinding* label

The learning tasks related to these findings and MEDUSA are inside two macro areas: **Classification** and **Similarity**.

Classification involves labeling the findings into predefined classes based on their content and context. Each finding is analyzed to assign appropriate labels from a predefined set. This is a classification task aimed at assigning one or more labels to every data point (finding) from a hierarchical set of classes—the so-called “tree of labels”. Here, the set of labels is structured as a rooted tree, hence allowing more context in classifying the labels. Any label can be under one or more parent labels, making this a multi-level classification system. This setting is called Hierarchical Multi-Label Classification (HMC). More than one label may be assigned at once to different levels of a hierarchical taxonomy. Taxonomy, in that context, refers to a structured classification system for items; it organizes them into nested categories or levels according to their relationships. HMC provides flexible and complete labeling of data points that reflect their multiple categories against this structured hierarchy.



For instance, a finding related to a financial discrepancy might be labeled under both “Financial Issues” and “Compliance Violations,” where each of these labels might have further subcategories. This ensures that all parent labels for a true label in the hierarchical structure are included.

3.2.1 Multi-Label Hierarchical Approach

Unfortunately, one of the most challenging tasks in machine learning is hierarchical multi-label classification because it requires the assignment of more than one label to data points, and these labels are hierarchically related. This added complexity comes because the model has to handle dependencies and relationships between the labels, structured hierarchically. This kind of classification is not only concerned with the correct labels but also considers and respects the hierarchy among them. Managing these intricate relationships and dependencies requires specialized strategies.

- **Bespoke Loss Functions:** One strategy is the design of bespoke loss functions tailored to the hierarchical structure of the taxonomy.[43] Unlike traditional loss functions, bespoke loss functions are specifically crafted to consider the hierarchy among labels. For example, in the MEDUSA system, a bespoke loss function can be designed to penalize misclassifications that occur higher up in the hierarchy more heavily than those at the leaf nodes. Meaning that missclassifications to the broader, more general categories—like “Financial Issues” are minimized, since such misclassifications are potentially more significantly detrimental to classification accuracy.
- **Hierarchical Layers in RNNs:** Another strategy involves using hierarchical layers within a recurrent neural network (RNN).[42] RNNs are suitable for processing sequential data like text by keeping some kind of memory about the previous inputs. By incorporating hierarchical layers, the network can learn dependencies between different levels of the hierarchy more effectively. For instance, in the MEDUSA system, lower layers of the RNN might focus on extracting sentence-level features, such as detecting key terms

and phrases. Higher layers then aggregate these features to understand document-level patterns and context.

- **Graph Convolutional Networks:** GCNs can be used to capture the hierarchical structure of the taxonomy in the design of the neural network itself. [25] By representing the hierarchy as a graph, GCNs can naturally incorporate the relationships between different labels. In this approach, a standard cross-entropy loss function can be used, but the network architecture ensures that the hierarchical dependencies are respected. For example, nodes in the graph represent labels, and edges represent the hierarchical relationships, enabling the network to propagate information in a way that mirrors the taxonomy.

For our scenario, designing a bespoke loss function that accurately captures the nuances of our hierarchical taxonomy may be the most effective approach but we will see which is the best solution.

3.2.2 Formal Definition

To provide a more detailed understanding of the hierarchical multi-label classification process, we introduce some formal definitions.

Let $\mathcal{F}$ be a set of findings (i.e. a collection of data points or instances that need to be analyzed and labeled) and let $T = (L, v_0, E)$ be a label set with labels L , root node v_0 and set of edges representing relationships between labels E . Each finding in $\mathcal{F}$ can have multiple labels from L .

The labeling relation $\mathcal{L}$ is a subset of the Cartesian product $\mathcal{F} \times L$. This means $\mathcal{L}$ pairs findings with their corresponding labels. We denote this relationship as $\mathcal{L}(F) = l_1, \dots, l_n$ if:

- $(F, l_i) \in \mathcal{L}$ for $i = 1, \dots, n$: The finding F is paired with each label l_i .
- $(F, l) \notin \mathcal{L}$ for all $l \notin l_1, \dots, l_n$: No other labels are paired with F
- $l_i \notin \mathcal{L}(l_j)$ for all $i \neq j$: Each label l_i is unique to F and not a subset of another label within F .

We define

$$\overline{\mathcal{L}(F)} := L(l) | l \in \mathcal{L}(F)$$

as the closure of labels of F . This means if a label l is assigned to F , and there are parent labels or more general categories above l , those parent labels are included in the closure.

We call finding classification any learning task that creates a series of label sets $\mathcal{P}_i$, which are the predictions of the classification. The goal is to have

$$\lim_{i \rightarrow \infty} \mathcal{P}_i = \mathcal{L}$$

according to a metric derived from a suitable loss function.

In the implementation of this classification task within MEDUSA, we use a dense output layer of size $|L|$ with sigmoid activation (Fig.3.7).

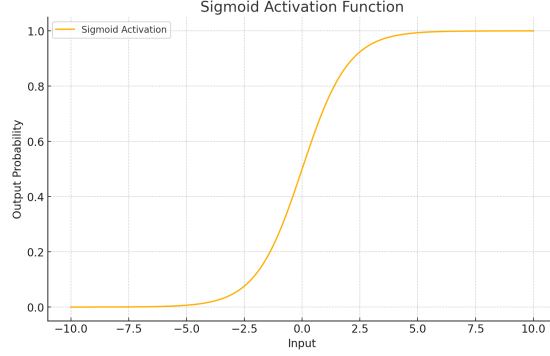


Figure 3.7: Sigmoid Activation Function

Finally, sigmoid activation in this final layer allows every independent label to have a probability assigned reflecting its presence or absence. This output architecture can be further tuned later on for better performance. The model was trained using the F1 score as the optimizing metric, with decreasing learning rates. With this, we ensure that our models are converging to some optimal solution, so we progress by slowly reducing their learning rates over time.

3.2.3 Loss Functions and Metrics

Typical binary classification metrics and loss functions need to be adjusted for our hierarchical structure of the label set. Cross-entropy alone cannot fit with nonindependent classes, as in our case, for certain problems.

To understand why, is necessary to give the formal definition of cross-entropy. For a finding set $\mathcal{F}$ with true labeling $\mathcal{L}$, the cross-entropy loss of a prediction $\mathcal{P}$ is defined as:

$$-\frac{1}{|\mathcal{F}|} \sum_{F \in \mathcal{F}} \chi_{\mathcal{L}}(F) \log \mathcal{P}(F) + (1 - \chi_{\mathcal{L}}(F)) \log(1 - \mathcal{P}(F)),$$

where $\chi_{\mathcal{L}}(\cdot)$ is the indicator function of $\mathcal{L}$, meaning $\chi_{\mathcal{L}}(F) = 1$ if F is true and 0 otherwise. $\mathcal{P}(F)$ is the predicted probability of F being true under $\mathcal{P}$. The loss is normalized by dividing by the size of the finding set to allow comparison across sets of different sizes.

While the cross-entropy loss does a good job in binary classification, the same fails in the case of HMC. This is due to the hierarchical dependencies existing between labels in hierarchical classification. The classes will not be independent, and misclassification at higher levels in the hierarchy may propagate errors down the tree, which causes big inaccuracies.

Before getting into hierarchical metrics, the foremost, very basic ideas of true positives, true negatives, false positives, and false negatives must be known (Fig.3.8).

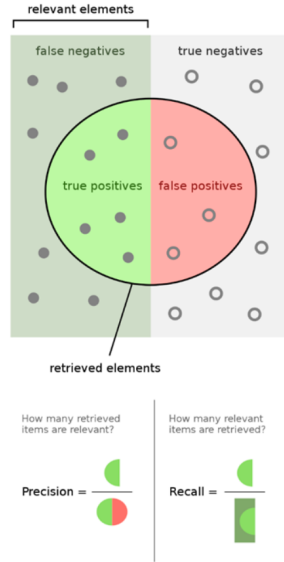


Figure 3.8: TP, TN, FP, FN

These form the basis for the most common metrics to measure the quality of binary classification:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2TP}{2TP + FP + FN}$$

However, again, in a hierarchical context, these metrics fall short because they treat all labels as equally distant from each other.

Example

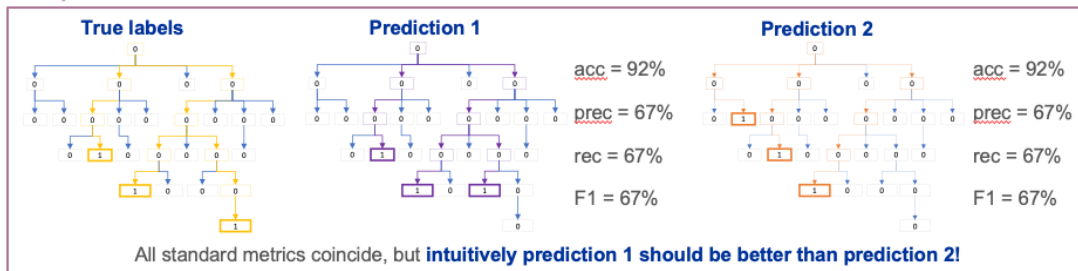


Figure 3.9: Standard Case

For example take in consideration Fig.3.9, and so consider three different predictions for the same set of true labels (Fig.3.9). Prediction 1 and 2 yield identical values for accuracy (92%), precision (67%), recall (67%), and F1 score (67%). Despite these identical standard metrics, intuitively, Prediction 1 should be considered better than Prediction 2. This discrepancy arises because the standard metrics do not consider the hierarchical dependencies between classes.

This will also show another problem of standard cross-entropy: when the set of labels is large and there are only a few true and predicted labels, the accuracy will always be high, independently of the prediction, because it is mainly driven by the number of true negatives.

To better capture the hierarchical nature of the classification task, we will need to adapt our loss functions and metrics. Here's how we can modify them:

Smoothing True Labels

We can “smooth” the true labels by turning the parents of correct labels into true positives. So from the previous example we obtain Fig.3.10.

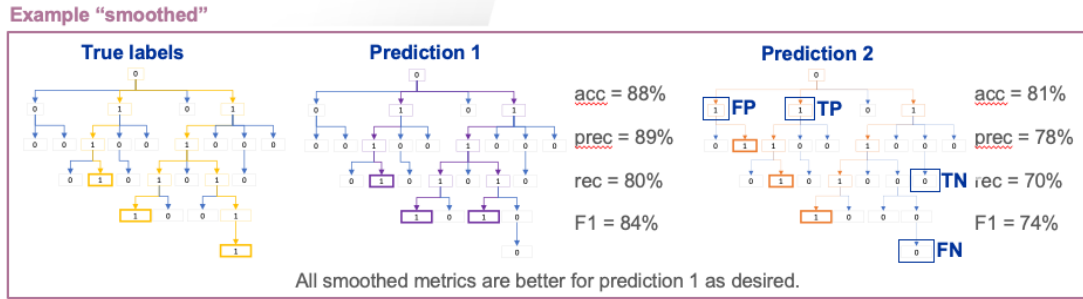


Figure 3.10: Smoothed

This way, the term $\chi_{\mathcal{L}}(F) \log \mathcal{P}(F)$ vanishes when F is not assigned the label. For labelled F it becomes minimal in absolute terms when also the prediction is true; i.e. when the labelled records are exactly the true positives. Similarly, $(1 - \chi_{\mathcal{L}}) \log(1 - \mathcal{P}(F))$ becomes minimal when the non-labelled records are exactly the true negatives.

Hierarchical True Positives, Negatives, False Positives, and Negatives

For a given finding F , we can count the number of nodes corresponding to true positives, true negatives, false positives, and false negatives. In order to avoid scaling problems, we divide by the total number of classes; although this divisor will cancel when considering the metrics. Given a finding F , for ease of notation we write its labels $\mathcal{L}(F)$ and predictions $\mathcal{P}(F)$ as $\{0, 1\}$ -valued functions. We can then define

$$TP(\mathcal{L}, \mathcal{P}, F) = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} \mathcal{L}(F)(l) \mathcal{P}(F)(l)$$

$$TN(\mathcal{L}, \mathcal{P}, F) = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} (1 - \mathcal{L}(F)(l)) (1 - \mathcal{P}(F)(l))$$

$$FP(\mathcal{L}, \mathcal{P}, F) = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} (1 - \mathcal{L}(F)(l)) \mathcal{P}(F)(l)$$

$$FN(\mathcal{L}, \mathcal{P}, F) = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} \mathcal{L}(F)(l) (1 - \mathcal{P}(F)(l))$$

When $\mathcal{L}$, $\mathcal{P}$ and F are clear from the context, we will omit them and simply write TP, TN, FP, FN .

Hierarchical Cross-Entropy Loss

Using the adjusted definitions of TP, TN, FP, FN and considering the smoothing process we can then define a new hierarchical cross-entropy loss function:

$$-\frac{1}{|\mathcal{F}||\mathcal{L}|} \sum_{F \in \mathcal{F}} \sum_{l \in \mathcal{L}} \mathcal{L}(F)(l) \log \mathcal{P}(F, l) + (1 - \mathcal{L}(F)(l)) \log(1 - \mathcal{P}(F)(l))$$

When $\mathcal{P} = \mathcal{L}$, i.e. the prediction matches exactly the labelling, accuracy = precision = recall = $F1 = 1$, as desired. The probability of any node is greater or equal than that of all its children guaranteed by the setup.

This loss function will then be consistent with the new concepts: TP, TN, FP, FN and the derived metrics. It gives predictions with probabilities increasing towards the root, reflecting our structure. For predicting, it will select all labels with a probability above a threshold. That is a rather conservative model to avoid mistakes. One rather aggressive approach would be to select the leaves with highest probabilities.

3.2.4 Finding Similarity

The second task of MEDUSA as stated is to calculate and finding Similarity. In our scenario finding similarity refers to any learning task that creates a series of similarity scores $simi$ on $\mathcal{F} \times \mathcal{F}$, such that $1 - simi$ is a metric on $\mathcal{F}$ and for all $F \in \mathcal{F}$ the ordering $G \succeq_i H$ defined by $simi(F, G) \geq simi(F, H)$ converges to an ordering corresponding to the predicate “is similar to F ”.

The theoretical definition is precise, but useless for a couple of reasons. First, it requires preference orders on the set of all possible findings from a large pool of experts and also a suitable metric to measure the convergence. Since we cannot get expert ratings on comprehensive sets nor the metrics, we use as an approximation the **Jaccard Index**:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- where A and B are two sets
- $|A \cap B|$ elements in the intersection A and B
- $|A \cup B|$ elements in the union A and B

This is used as a proxy for the similarity score. The set of findings serves as our training set. As more user interactions occur, eventually the possibility to collect additional information will be there to develop more meaningful semantic models that stand on their own.

For practical realization of the application, we did not rely on a custom model of similarity. Rather, we derive similarity from the classification model by extending it to text embedding. We map the texts to a 350-dimensional vector space:

$$\{\text{Texts}\} \rightarrow \mathbb{R}^{350}$$

Finding 6	The mapping of trades to the portfolio hierarchy exhibits deficiencies	
Legal Requirement	Articles 367(1)(a) and 367(2)(e) of the CRR	
Criteria	According to Article 367(1)(a) of the CRR, the IMA shall capture accurately ...	
Condition	The controls ensuring that full revaluation books and sensitivities are mapped to the portfolio hierarchy exhibit the following deficiencies ...	
Root Cause	Books may fail to get mapped to the hierarchy for different reasons...	
Effect	The lack of clear criteria for remediation of unmapped trades, the backlog of unmapped books and the non-consideration as part of the RINV process may lead to...	
Severity	F3	

$\mapsto$

Label	Probability
#1	0.000034729
#2	0.000143795
⋮	⋮
#350	0.000002953

Figure 3.11: Example of 350-dimensional space mapping

The Fig.3.11 can help to understand better. The choice of 350 dimensions comes with a trade-off of dimensionality that must be high enough to obtain enough information in the embeddings and low enough to keep computational efficiency in line. High-dimensional embeddings provide a rich representation of the text data, improving the model’s ability to understand and classify it accurately.

We will then define the similarity of two datapoints, findings, as the inverse of their shifted $L1$ distance under this embedding:

$$s(\mathbf{f}_1, \mathbf{f}_2) = \frac{1}{(1 + \|P(\mathcal{L}_1) - P(\mathcal{L}_2)\|_1)}$$

This is, in turn, very efficient and allows for computing similarities with a full database by using only pre-calculated embedding tables.

Practical Choices and Implementation

There is a large number of possible choices for defining the distance in the vector space and how to turn the distance into a similarity score. In the absence of stringent conceptualization of “*similarity*”, we can make the simplest possible choices, and revisit them using user feedback.

Our choice ensures that $s(f, f) = 1$, meaning identical findings have a similarity score of one. On the other end of the spectrum, the minimum similarity is determined by the $L1$ diagonal of the unit cube in $\mathbb{R}^{350}$, so $s(f, g) > 0.0028$ for all f, g .

These techniques guarantee that the scores of similarity will be real and meaningful; they, therefore, offer a good platform on which further improvements can be made. It is the adoption of this approach that gives the MEDUSA system the capability to track and compare similar findings with a remarkably high level of effectiveness, improving the system’s ability in the delivery of insightful and accurate classification.

3.2.5 Severity Prediction as Regression Learning Task

Another functionality of MEDUSA was severity prediction. In the absence of some systematic way of identification of severity, the critical issues may get overlooked or not addressed in a timely manner. Hence, by predicting the severity

of findings, MEDUSA would help in streamlining the supervisory process so that high-severity issues get proper attention. Here, severity prediction is treated as a regression learning task.

To formalize this, we define two functions for mapping real numbers to the constrained set of severity levels:

- **Constrained Smallest Integer Function:**

$$\lfloor x \rfloor_1^4 = \min(\max(\lfloor x \rfloor, 1), 4)$$

This function rounds down to the nearest integer within the range 1 to 4.

- **Constrained Closest Integer Function:**

$$\lfloor x \rceil_1^4 = \min(\max(\lfloor x \rceil, 1), 4)$$

This instead is responsible for rounds to the nearest integer within the range 1 to 4.

The goal of severity regression is to produce a series of predictions of real numbers s_i (or their constrained equivalents $\lfloor s_i \rfloor_1^4$ or $\lfloor s_i \rceil_1^4$) such that the sum of distances between the true severities and predicted severities decreases over time, ideally converging to 0. This can be expressed mathematically as:

$$\sum_{F \in \mathcal{F}} d(s(F), s_i(F))$$

where $d(s(F), s_i(F))$ is a suitable distance metric derived from the loss function.

Implementation in MEDUSA

For the implementation in MEDUSA, a single node output layer with ReLU (Rectified Linear Unit) (Fig.3.12): activation is used.

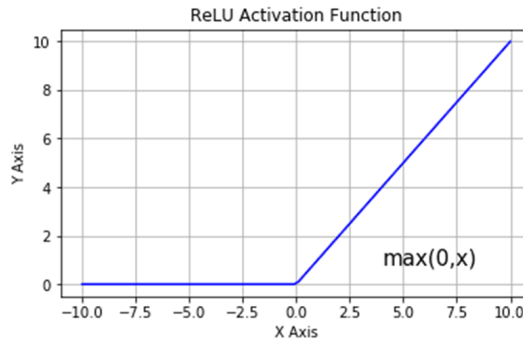


Figure 3.12: ReLU Activation Function

ReLU is chosen because it effectively handles the regression task by allowing the network to learn non-linear relationships while ensuring non-negative outputs, which aligns with the natural number severity scale.

For clarity, let's make an example. Consider a set of findings $\mathcal{F}$ with true severities $s(F) \in \{1, 2, 3, 4\}$. The regression model predicts a severity score for each finding. If for example the model predicts $s_i(F) = 2.7$ for a finding, the constrained closest

integer function $\lfloor s_i(F) \rfloor_1^4$ would round it to 3.

Over time, by using the loss function, the model learns to minimize the error between the predicted severities and the true severities. This process continues until the sum of the distances between $s(F)$ and $s_i(F)$ converges towards 0, indicating finally accurate severity predictions.

3.2.6 Handling of Long Findings

Since we're going to use the BERT Transformer model in our project, many limitations of the Transformer models will have to be kept in mind.

One important point is that all the transformer models, similar to most machine learning models, have a maximum length of tokens supported for each instance in the training, development, or test set. In such a case, when the maximum length is overshoot, truncation of the sequence to allowable length is the most common practice. This could, however, lead to loss of vital information when it is done, especially on relevant information in the MEDUSA system whereby the most relevant information about a finding is completely omitted. Several alternatives exist that can mitigate this, and they can be combined for better results. Here is an overview of the one we proposed.

Use trained models and allow longer sequences

A solution would be to utilize the trained models supporting longer sequences. In this way, information loss would be reduced to a minimum because only those sequences longer than the already extended maximum length would be truncated.

Skip Long Cases

Another is the skipping of cases that are longer than the maximum allowed length. This strategy would be applicable at the model fine-tuning phase and not during the prediction of new cases. This will require that such new cases to be predicted be restricted to the maximum length so that the tool can process them. However, if the number of long cases in the dataset was high, this process would lose too much information. For this reason, such a solution would not be very practical either.

Truncate long findings

Easiest to engineer will be truncating long findings at their supported length, as most machine learning methods support this. However, this can result in sub-optimal results as some very pertinent information may be lost. Truncation may affect differently depending on how the model processes each finding. For example, different ways to order fields while concatenating will yield very different results.

A more sophisticated method of truncation is to eliminate some of the fields in a finding that hold lower relevance. Knowledge from the domain is, however, required to know which fields are of lower priority. Even though this technique could not fully reduce loss of information, it is likely to work better than arbitrary truncation of all cases.

3.3 A Custom Huggingface Pipeline

Having explored the theoretical approach, we now transition from theory and formulas to practical implementation. This section will demonstrate how to apply these concepts using the Hugging Face library. We will provide Python code snippets to highlight the most important and critical aspects of the implementation.

3.3.1 The Huggingface Library

Of the several solutions that were contemplated at the beginning, Hugging Face turned out to be the most feasible platform for both pre-trained models and user-friendly tools. One of these abstractions, which makes these tasks effortless, is the Hugging Face pipeline. This interface provides an easy way to load a model, preprocess data, and make inferences at a high level.

This section outlines the process of setting up and using a custom Hugging Face pipeline tailored to MEDUSA's needs.

Tokenization and Model Preparation

Tokenization as already mentioned is a crucial step in NLP, where text data is converted into tokens, smaller chunks that the model can understand. For our pipeline, we use the pre-trained BERT tokenizer.

Here's how BERT tokenization integrates into MEDUSA:

```
1 from transformers import AutoTokenizer
2 import numpy as np
3
4 # Initialize the tokenizer
5 tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
6
7 # Tokenize the dataset
8 tokenized_data = tokenizer(dataset["sentence"], return_tensors='np',
9                             padding=True)
10
11 # Convert the tokenized data to a dictionary for compatibility with
12 # Keras
13 tokenized_data = dict(tokenized_data)
14
15 # Prepare the labels
16 labels = np.array(dataset["label"]) # Assuming labels are already
17 # in array form (0 and 1)
18
19 # Load the pre-trained BERT model
20 model = TFAutoModelForSequenceClassification.from_pretrained
21     ("bert-base-cased")
22
23 # Compile the model with an Adam optimizer and a low learning rate
24 model.compile(optimizer=Adam(3e-5)) # Note: No loss argument is
25 # specified here
```

Main steps are:

- Initialization: We begin by initializing the BERT tokenizer. This tokenizer splits the text into words, subwords, or characters and converts these into numerical IDs that the model can process.

- Tokenization: The tokenizer processes the text data, breaking it into tokens and converting these tokens into numerical representations that the model can handle.
- Padding: To ensure that all tokenized sequences are of the same length, we pad shorter sequences. This step is essential for efficient batch processing, enabling a model to process many sequences at a time.

Model Training

Now that the model and data are ready, train. In the course of training, it will learn to map input texts onto their target labels. This is done by tuning the weights of the model so that the predicted labels have the least difference with the actual labels present in the training data. This step is therefore very important for MEDUSA to learn from the specific textual data it comes across to improve in its classification accuracy.

```
1 # Train the model
2 model.fit(tokenized_data, labels)
```

Evaluation

Evaluation is part of the development process of a model, allowing one to get a measure of performance of a working model so as to make the necessary adjustments. In MEDUSA, we use the accuracy metric to evaluate the model. In specific in order to evaluate the model's performance, we use the `evaluate` library to compute the accuracy of our predictions. The `compute_metrics` function processes the model's predictions and computes the accuracy score.

```
1 import evaluate
2 import numpy as np
3
4 # Load the accuracy metric
5 accuracy = evaluate.load("accuracy")
6
7 # Define the function to compute metrics
8 def compute_metrics(eval_pred):
9     predictions, labels = eval_pred
10    predictions = np.argmax(predictions, axis=1)
11    return accuracy.compute(predictions=predictions,
12                           references=labels)
```

3.3.2 MEDUSA Pipeline

MEDUSA is then provided with good, well-structured training and inference pipelines that deal efficiently with text classification tasks. They are based on standard practices but are fine-tuned to leverage the capabilities of the Hugging Face Transformers library previously discussed. The Fig 3.13 try to show these steps. Each of these will be described in later sections.

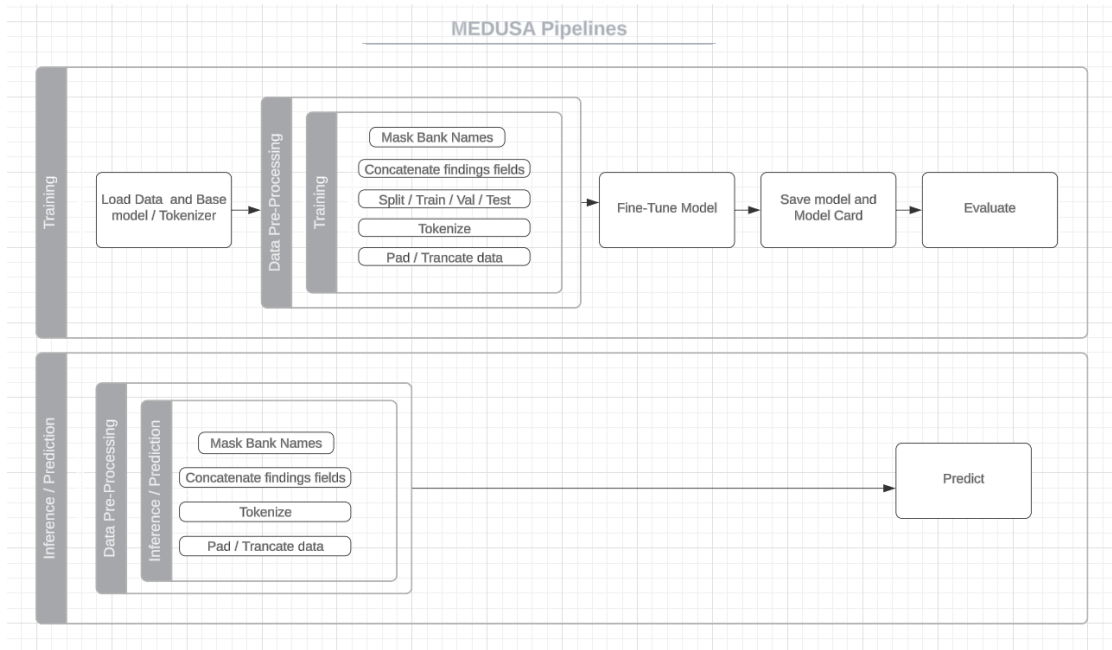


Figure 3.13: Medusa Pipelines

Training Pipeline

MEDUSA involves several steps in model preparation, training based on the dataset, and testing the performance with respect to characteristics. Detailed breakdown of the steps is as follows:

1. Load Data and Base Model/Tokenizer:

- *Loading Data:* The dataset is first loaded, containing a text input and its corresponding label. This constitutes the core of the training dataset.
- *Loading Model/Tokenizer:* Simultaneously, a pre-trained model and tokenizer are loaded from the Hugging Face model hub. The “*bert-base-cased*” model is used. The tokenizer is responsible for converting text into tokens that the model can process.

2. Pre-process Data:

- *Tokenization:* The text data is tokenized using the pre-trained tokenizer.
- *Padding and Truncation:* These tokenized sequences are either padded or truncated to be of equal length. In this way, processing can be parallelized in batches, whereby the model processes many sequences at a time.

3. Fine-tune Model:

- In fine-tuning, weights of the pretrained model are adjusted in accordance with the dataset so that it fits better to make predictions. In the process, it will learn to associate the tokenized text data with the correct labels. Fine-tuning is, hence, very important in adapting the general pre-trained model to the exacting subtleties in the new dataset.

4. Evaluate:

- Finally, the model is evaluated for performance using accuracy, precision, recall, F1 score, among other metrics, to know how well it is doing and further adjustments that may be needed. Evaluation is performed on a separate validation set to ensure generalizability.

5. Save Model and Model Card:

- *Saving Model*: Once the model has been trained and evaluated, it is saved for future use. This includes saving the model weights, configuration, and tokenizer.
- *Model Card*: A model card should be created for the proper documentation of the model with regards to the intended use, performance metrics, training data, and other ethical considerations. This is important documentation for transparency and reproducibility, but also for users who would like information about the strengths and limitations of this model.

Inference/Prediction Pipeline

The inference pipeline in MEDUSA is designed for making predictions on new, unseen data. It is simpler and more streamlined than the training pipeline. Here are the steps involved:

1. Pre-process Data:

- *Tokenization*: Similar to the training pipeline, the new text is tokenized with the same tokenizer used during training.
- *Padding and Truncation*: These tokenized sequences are either padded or truncated to be of the exact length expected by the model. This is to ensure that the model processes the new data efficiently.

2. Predict:

- The pre-processed data are fed into a fine-tuned model for generating its predictions. It outputs probabilities or class labels for each input sequence in a classification role. Final processing on the output is then made to get the labels that go with the input data.

3.3.3 Data pre-processing

Pre-processing for training and inference in MEDUSA involves similar steps to ensure consistency, although there are slight differences to accommodate the specific needs of each phase.

Training Data Pre-processing

1. Mask Bank Names:

- Sensitive information such as bank names is masked to ensure privacy and data security. This step involves replacing actual bank names from ARs with placeholder tokens.

2. Concatenate Finding Fields:

- Different fields corresponding to the findings are all concatenated into one text string.

3. Split Train/Val/Test Sets:

- The dataset is split into training, validation, and test sets. This split allows for effective training, validation, and evaluation of the model. Typically, the data is divided in a ratio such as 70% for training, 15% for validation, and 15% for testing. [12]

4. Tokenize:

- The consolidated text data is tokenized using the pre-trained tokenizer. Tokenization converts the text into tokens that the model can process.

5. Pad/Truncate Data:

- The tokenized sequences are padded or truncated to ensure uniform length. This step is crucial for efficient batch processing during model training.

Inference/Prediction Data Pre-processing

1. Mask Bank Names:

- Similar to the training phase, any sensitive information such as bank names is masked to maintain privacy and data security.

2. Concatenate Finding Fields:

- Fields related to the new findings are concatenated into a single text string to provide comprehensive input for the model.

3. Tokenize:

- The concatenated text is tokenized using the same tokenizer used in the training phase.

4. Pad/Truncate Data:

- The tokenized sequences are padded or truncated to match the expected input length for the model.

3.3.4 Fine-Tuning

The most important step of any pre-trained model, toward any specific task or dataset, is fine-tuning. Fine-tuning in MEDUSA will basically mean training the pre-trained BERT model on our dataset so that it changes the weights to fit better predictions. Therefore, the model will have learned the particularity of the new data and become highly accurate with reliability for the task at hand. It simply fine-tunes general skills brought in by the pre-trained BERT model, with the specifics pertinent to our classification task. The same thing would be realized as

follows using basic steps.

Define parameters to be used in guiding training: model definition, dataset to train and validate on, number of epochs, learning rate, loss function, metrics of evaluation, dropout. All these parameters will impact the shape of training dynamics and ensure that the model learns well.

These will be the subsets of the specific data that MEDUSA will deal with: the training and validation datasets. This is further divided into subsets allowing both learning and evaluation during training.

The number of epochs controls the frequency with which the entire training dataset is passed through the model, and the learning rate controls how large the steps are when updating the model's weights. It measures something about how different model predictions are from their corresponding true labels; thus, in some sense, it drives the optimization process. Accuracy, precision, recall, and the F1 score are more illuminating metrics of model performance.

Dropout rate: This is the method of preventing overfitting whereby a fraction of input units is randomly set to zero when training a model.

Now that we've set our parameters, let's compile the model. We shall define an optimizer, loss function, and evaluation metrics here. We're going to use Adam for optimization since it really gives good efficiency versus effectiveness when it comes to updating model weights. We then compile the model to make sure everything is correct about configurations during the training process.

In details, the Adam optimizer provides solid performance and ease of use during training in deep learning models. It combines the advantages of two other extensions of stochastic gradient descent: Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp) [24].

AdaGrad is targeted at adaptively modifying the learning rate for every parameter on its own by its inverse proportion to the square root of the sum of all squares of historical gradients for this parameter. This ensures that a parameter whose value gets frequently changed gets smaller updates, while the ones that do not get changed frequently get larger updates, allowing an optimizer to perform well in scenarios with sparse gradients. On the other hand, RMSProp also adapts the learning rate of every parameter but looks at recent gradients instead of summing up all the gradients in the past. This approach applies by maintaining a moving average of the squared gradients as the base for controlling the learning rate to avoid the diminishing learning rates problem in AdaGrad. Thus, RMSProp helps to stabilize the process of learning in the non-stationary setting where the learning rate may turn extremely small.

```
1 def fine_tune_model(model, loss, train_dataset, val_dataset=None,
2   metrics=None, num_epochs=10, learning_rate=5e-5, dropout=0.1):
3     # Set dropout rate if specified
4     if dropout is not None:
5         model.config.hidden_dropout_prob = dropout
```

```

6     # Use default metrics if none are provided
7     if metrics is None:
8         metrics = []
9
10    # Compile the model with specified optimizer, loss, and metrics
11    model.compile(
12        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate
13        ),
14        loss=loss,
15        metrics=metrics,
16        run_eagerly=True # Enable eager execution for debugging
17        purposes
18    )
19
20    # Train the model on the training dataset
21    results = model.fit(train_dataset, epochs=num_epochs,
22        validation_data=val_dataset, verbose=1)
23
24    # Return the training results
25    return results

```

Model training is the most central step in tuning. During this stage, the model ingests the training data, looking to modify its weights so that it minimizes loss. It requires multiple epochs, in which the model learns iteratively to make better predictions. All this time, it monitors model performance and avoids overfitting using validation data.

```

1  # Train model
2  training = fine_tune_model(model=model, train_dataset=train_dataset,
3      val_dataset=val_dataset,
4      num_epochs=num_epochs,
5      learning_rate=learning_rate,
6      loss=loss, metrics=metrics,
7      dropout=dropout)

```

Predict function factory

In this regard, accurate predictions from the output of the model in the MEDUSA system are critical. A specific function is created to do this. The `predict_from_logits_func_factory` function creates a prediction function that changes the raw logits; this is what the neural network's final layer will spit into meaningful predictions that include probability scores with their corresponding labels. Here is the complete code for the `predict_from_logits_func_factory`:

```

1  def predict_from_logits_func_factory(id2label_dict,
2      output_layer_func=tf.nn.sigmoid, probability_threshold=0.3,
3      min_num_preds=0, max_num_preds=None):
4
5      # Define a function to generate predictions from logits
6      def predict_from_logits_func(logits):
7
8          """Return a prediction from a tensor of logits."""
9
10         nonlocal max_num_preds
11         nonlocal min_num_preds
12         nonlocal probability_threshold
13         nonlocal output_layer_func
14
15         # Apply the output layer function (e.g., sigmoid) to the
16         logits

```

```

12     probs = output_layer_func(logits)
13
14     # Set max_num_preds to the number of classes if not specified
15     if max_num_preds is None:
16         max_num_preds = logits.shape[-1]
17     # Ensure min_num_preds is set
18     if min_num_preds is None:
19         min_num_preds = 0
20
21     # Select the top k predictions based on probability
22     selection = tf.math.top_k(probs, k=max_num_preds,
23                               sorted=True)
24     counts = tf.math.count_nonzero(input=selection.values >=
25                                   probability_threshold, axis=-1)
26     num_clones = tf.math.maximum(tf.math.minimum(counts,
27                                                   max_num_preds), min_num_preds)
28
29     # Generate the prediction indices
30     pred_indices = [list(tf.numpy_function(np.sort,
31                                           [np.random.choice(selection.indices[tf.math.minimum(i,
32                                                                 counts[i])], num_clones[i])], tf.int64)) for i in
33                     range(logits.shape[0]))]
34     # Get the prediction probabilities
35     pred_probs = [[logits.numpy()[i][j] for j in
36                   pred_indices[i]] for i in range(logits.shape[0])]
37     # Map the prediction indices to labels
38     pred_labels = [[id2label_dict[j] for j in pred_indices[i]]
39                    for i in range(logits.shape[0])]
40     # Create label indicators
41     label_indicators = [[1 if i in pred else 0 for i in
42                          range(logits.shape[1])] for pred in pred_indices]
43
44     # Return the label dictionary, label indicators,
45     probabilities, and labels
46     return id2label_dict, label_indicators, pred_probs,
47           pred_labels
48
49 return predict_from_logits_func

```

The `predict_from_logits_func_factory` function generates a customized prediction function based on given parameters. This function allows for flexible configuration to meet different prediction requirements, such as setting probability thresholds and limiting the number of predictions.

The function begins by defining parameters including `id2label_dict` (a dictionary mapping label IDs to human-readable labels), `output_layer_func` (a function to convert logits into probabilities, defaulting to sigmoid), `probability_threshold` (the minimum probability required for a label to be considered in the predictions), `min_num_preds` (the minimum number of predictions), and `max_num_preds` (the maximum number of predictions).

When the `predict_from_logits_func` inner function is called, it first applies the `output_layer_func` to the logits to obtain probabilities. This transformation converts raw model outputs into interpretable probability scores. If the `max_num_preds` or `min_num_preds` are not provided, the function assigns default values to handle any input shape and constraints.

The function then uses `tf.math.top_k` to select the top `max_num_preds` probabilities for each input sample, focusing on the most likely predictions. It counts how many of these top probabilities exceed the `probability_threshold` and ensures the number of predictions is within the specified range using `tf.math.maximum` and `tf.math.minimum`.

It randomly selects top indices of probabilities for each sample. In the case where there are multiple indices with similar probabilities, it would result in diversity in the predictions. Extract the corresponding probabilities and labels for these indices, then generate binary indicators for each label to help in further processing or evaluation.

It finally returns the dictionary mapping between label IDs and labels, binary indicators of the presence of labels, probabilities of selected labels, and the labels themselves.

Custom loss function

Going back, also we recap that the custom loss function is critical in tailoring the training process to our specific needs, as we said already. We cannot expect the standard loss functions to fit all the intricacies of the data and the labels being used in the MEDUSA system always. The `simple_custom_loss_factory` function creates the custom loss function above, which will be used to incorporate a number of requirements into the training process. This allows the model to take into account things such as hierarchical label structures and customized probability transformations during training.

Function Definition and Parameters The function takes the following parameters:

- `id2label_dict`: A dictionary mapping label IDs to human-readable labels. This dictionary plays a huge role in mapping the indices of a model's output to meaningful labels.
- `checkpoint`: An optional parameter for loading presaved label mappings, in case they are already defined somewhere else.
- `output_layer_func`: A function to turn logits raw output scores from the model into probabilities. By default, this is the sigmoid function.

If both `id2label_dict` and `checkpoint` are not provided, the function defaults to using the `labels_master.get_label_ids()` method to generate the dictionary. This dictionary maps each label index to its corresponding label, ensuring the outputs are interpretable.

Inner Custom Loss Function At the core of this factory function is the `simple_custom_loss` function, which does exactly what the name suggests: it calculates loss between the ground truth labels (`y_true`) and the predicted labels (`y_pred`). In more detail, this does the following:

1. **Output Layer Function:** Passes the predicted logits through the output layer func to transform them into probabilities. This step is for transforming the raw outputs into interpretable probability scores.
2. **Fetch True Label IDs:** The true labels (`y_true`) are mapped to their corresponding label IDs through the `id2label_dict`. This step is taken so that the labels will be in one standard format for further processing.
3. **Compute Label Closures:** For every true label, this function fetches its closure along with a set of related labels using method `labels_master.get_closure`. Again, this is very important in handling hierarchical or grouped labels to make sure that related labels are dealt with jointly.
4. **Convert True Labels to Binary Format:** True labels are changed into binary format in which 1 means the label is part of the true label closure and 0 otherwise. This binary representation will be required while computing the loss.
5. **Calculate Loss Components:**
 - **Summand 1:** This is the negative log likelihood of the true labels.
 - **Summand 2:** This, in turn, is the log likelihood of the false labels.
6. **Combine Loss Components:** These two summands are added and normalized by the number of predictions to obtain the final loss. In this way the loss is appropriately scaled.

Finally, the `simple_custom_loss_factory` function returns the `simple_custom_loss` function ready for use during model training.

Here is the complete code for the `simple_custom_loss_factory`:

```

1 def simple_custom_loss_factory(id2label_dict=None, checkpoint=None,
2   output_layer_func=tf.nn.sigmoid):
3     # Initialize id2label_dict if not provided and no checkpoint is
4     given
5     if id2label_dict is None and checkpoint is None:
6         id2label_dict = {k: v for k, v in
7             enumerate(labels_master.get_label_ids())}
8
9     # Placeholder for further processing if only checkpoint is
10    provided
11    if id2label_dict is None:
12        pass
13
14    # Define the custom loss function
15    def simple_custom_loss(y_true, y_pred):
16        # Apply the output layer function (e.g., sigmoid) to the
17        predictions
18        y_pred = output_layer_func(y_pred)
19
20        # Map true labels to their corresponding label ids
21        y_true_label_ids = [[id2label_dict[idx] for idx, val in
22            enumerate(y_val)] for y_val in y_true]
23
24        # Get the closures for the true labels

```

```

19     y_true_closures = [labels_master.get_closure(ids) for ids in
20                         y_true_label_ids]
21
22     # Create the true label matrix
23     y_true = [[1 if label in y_true_closure else 0 for label in
24                 id2label_dict.values()] for y_true_closure in
25                y_true_closures]
26     y_true = tf.cast(y_true, tf.float32)
27
28     # Calculate the loss components
29     summand_1 = tf.multiply(y_true, -tf.math.log(y_pred))
30     summand_2 = tf.multiply(y_true - 1, tf.math.log(1 - y_pred))
31
32     # Compute the final loss value
33     return tf.reduce_sum(tf.divide(tf.add(summand_1, summand_2),
34                                     tf.constant(y_pred.shape[0], dtype=tf.float32)))
35
36 return simple_custom_loss

```

Here is a step-by-step breakdown to make it a bit clearer:

- **Initial Setup:** First, it checks whether `id2label_dict` and `checkpoint` are given. In case of absence, it creates a default label dictionary by means of `labels_master.get_label_ids()`. This dictionary will be mapping model outputs to meaningful labels.
- **Output Layer Function:** The predicted logits (`y_pred`) are converted into probabilities through the `output_layer_func`. This is a step to be taken in converting the raw model outputs into a form from which loss can be computed.
- **Mapping True Labels:** The true labels are mapped to their corresponding label IDs thanks to `id2label_dict`.
- **Calculating Label Closures:** Compute the closure of all true labels; that means all labels associated with any given label. (Required because of hierarchical relationships between the labels.)
- **Binary Conversion:** The above-mentioned true labels would be then converted into a binary format where each label is represented as 1 if it falls within the closure of the true label; otherwise, it is represented as 0. It requires binary representation for the loss computation.
- **Loss Calculation:** This loss would be the sum of two components: negative log likelihood of the true labels and log likelihood of the false labels. These two components will then be combined and normalized to give the final loss value.

It is through the inclusion of these steps that the `simple_custom_loss_factory` function guarantees that during training, MEDUSA considers the structure and relationships of labels as has been previously discussed.

Custom metrics

Coming back to custom metrics, these will be essential in evaluating our model performance in a form consistent with specific application requirements. We have said

earlier that standard metrics can lack nuance to capture the intricacies of data or particular goals of the MEDUSA. The `SimpleMetric` and `SimpleAccuracy` classes provide an infrastructure for easily defining and using such custom metrics as necessary.

More in details, the `SimpleMetric` class is a base class for creating custom metrics in TensorFlow. It extends `tf.keras.metrics.Metric` and provides a structure for defining how the metric should be computed.

Class Definition and Initialization The `SimpleMetric` class takes the following parameters:

- `name`: The name of the metric.
- `predict_func`: A function that processes the model's predictions (logits) and converts them into interpretable labels and probabilities.

The `__init__` method initializes the metric, setting the `predict_func` and other necessary parameters. If the `predict_func` is not provided, it raises a `Value Error` because the metric cannot function without it.

Updating the Metric State The `update_state` is called during the evaluation of a model and will update the state of the metric based on true labels, (`y_true`) and predicted logits (`y_pred`). Here's how it works:

1. **Predict Function:** The `predict_func` is applied to `y_pred` to get the processed predictions like label indicators, predicted probabilities and predicted labels.
2. **True and Predicted Labels:** The true labels are mapped to their corresponding label IDs, and the closure of them is retrieved using the method `labels_master.get_closure`. In the same way, closures for the predicted labels are retrieved.
3. **Binary Conversion:** The true and the predicted labels should be changed into a binary format in which 1 is correspondent with each label belonging to the true or predicted label closure, while the rest should be 0.
4. **Calculating True Positives:** The measure of true positives is evaluated by computing a logical AND of the true and the predicted labels, returning a tensor with a value of 1 only in the case when both of them are True. Then, it sums up these values and adds this sum to the state of the metric.

Here is the code for class `SimpleMetric`:

```
1 class SimpleMetric(tf.keras.metrics.Metric):
2
3     def __init__(self, name, predict_func=None, **kwargs):
4         """Constructor."""
5         super(SimpleMetric, self).__init__(name=name, **kwargs)
6         self.predict_func = predict_func
7
8     def update_state(self, y_true, y_pred, sample_weight=None):
9         """Update the state of the metric during the evaluation of a
            model."""
```

```

10         if self.predict_func is None:
11             raise ValueError("Predict function must be provided")
12
13         id2label_dict, label_indicators, pred_probs, pred_labels =
            self.predict_func(y_pred)
14
15         # True values and predictions complemented by labels in the
            closures
16         y_true_label_ids = [[id2label_dict[idx] for idx, val in
            enumerate(y_val)] for y_val in y_true]
17         y_true_closures = [labels_master.get_closure(ids) for ids in
            y_true_label_ids]
18         y_pred_closures = [labels_master.get_closure(pred_labels)
            for pred in pred_labels]
19         y_true = [[1 if label in y_true_closure else 0 for label in
            label_set] for y_true_closure in y_true_closures]
20         y_pred = [[1 if label in y_pred_closure else 0 for label in
            label_set] for y_pred_closures in y_pred_closures]
21
22         y_true = tf.cast(y_true, tf.bool)
23         y_pred = tf.cast(y_pred, tf.bool)
24
25         # True positives
26         values = tf.logical_and(tf.equal(y_true, True),
            tf.equal(y_pred, True))
27         self.true_positives.assign_add(tf.reduce_sum(tf.cast(values,
            self.dtype)))

```

Based on SimpleMetric, class SimpleAccuracy implements an accuracy metric specific to MEDUSA. It inherits class SimpleMetric and computes accuracy by means of true-positives, true-negatives, false-positives and false-negatives.

Class Definition and Initialization The class SimpleAccuracy is initialized with the same arguments as SimpleMetric like name and predict _func. It initializes the state variables, which will be used in the accuracy computation.

Calculating Accuracy Calculating Accuracy It uses the result method to calculate the accuracy depending on the count of true positives, true negatives, false positives, and false negatives. This returns the ratio of correct predictions against total predictions. Here's the code for the class SimpleAccuracy:

```

1 class SimpleAccuracy(SimpleMetric):
2     """Class that inherits from SimpleMetric and implements accuracy
            as...."""
3
4     def __init__(self, name='simple_accuracy', predict_func=None,
            **kwargs):
5         """Constructor."""
6         super(SimpleAccuracy, self).__init__(name=name,
            predict_func=predict_func, **kwargs)
7
8     def result(self):
9         """Get the result of the accuracy following the simple
            method."""
10        return (self.true_positives + self.true_negatives) /
            (self.true_positives + self.true_negatives +
            self.false_positives + self.false_negatives)

```

The class `SimpleMetric` provides a baseline for implementing custom metrics. It handles updating and maintaining metric states in the course of model evaluation to ensure appropriate handling of predictions and true positive counts.

While the `SimpleAccuracy` class stretch this functionality to include true negatives, false positives, and false negatives, so it will be an all-rounded accuracy metric for correctness of predictions.

3.3.5 TensorFlow vs. PyTorch

By mentioning deep learning frameworks, a thought certainly falls on the two most popular ones: TensorFlow and PyTorch. Both are supported by Hugging Face, and give examples and tools to use pre-trained models in either framework. As of now, MEDUSA is realized using TensorFlow, given its high flexibility and the robustness of the ecosystem but since in several related work PyTorch is used is useful to make a comparative analysis.

TensorFlow Execution Modes

There are two major modes of execution in TensorFlow, each with their associated benefits and use cases.

- **Eager Execution:** In eager execution mode, operations are evaluated immediately as they are called from Python.[7] This mode is quite intuitive, and thus, it is easy to debug. It's more suitable for development and experimentation. It helps developers easily write and debug code, much the same as standard Python code is written and run. When you conduct a tensor operation in eager mode, you get results immediately, which makes it easier for debugging.
- **Graph Mode:** Graph mode, on the other hand, constructs a computation graph that represents the sequence of operations to be executed.[5] This mode is very fast and efficient and also allows high parallelism and chances of optimization. This is really beneficial when large models have to be trained and deployed to production environments. Graph mode is more difficult to learn, harder to use. Debugging isn't as intuitive since the graph has to be built before it can execute, making tracing and fixing errors more difficult. When you define a model in graph mode, a computation graph is constructed that will execute all at once, optimizing the performance, but making debugging much more complex.

PyTorch Execution Mode

On the other side, PyTorch works under a system called Dynamic Computation Graphs—the same as “*eager mode*”. It is somewhere in between TensorFlow's eager execution and graph mode. Dynamic Computation Graphs are built at every iteration on the fly. This gives flexibility and ease of debugging, yet still yielding not-so-bad performance.

Dynamic Computation Graphs: PyTorch's dynamic graphs are more flexible and easier to debug than TensorFlow's graph mode. [30] They are created and run dynamically, and this therefore allows for instant feedback during runtime. Particularly, it is this that makes PyTorch very dominant in research and experimental contexts. Although this is generally slower than static graphs like TensorFlow's graph mode, dynamic graphs are faster than TensorFlow's eager execution.

Comparison and Contextualization in MEDUSA

The choice between TensorFlow and PyTorch often depends on the specific requirements of the project and the preferences of the development team. Below is a comparison of the default modes for TensorFlow and PyTorch:

Framework	Default Mode
TensorFlow 1.x	Graph (Eager possible)
TensorFlow 2.x	Eager (Graph possible)
PyTorch	Dynamic Computation Graphs
Transformers (TF 2.x)	Graph (Eager possible)

MEDUSA uses Tensorflow 2.x, and eager execution is preferred during the development and experimentation stages due to its simplicity and the ease of debugging. Again, graph mode is most often used in final training and deployment to reap the benefits of efficiency and performance.

Below is a sample code snippet showing how to compile a model with eager execution on in TensorFlow for easy debugging:

```
1 model.compile(  
2     optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),  
3     loss=loss,  
4     metrics=metrics,  
5     run_eagerly=True  
6 )
```

By setting `run_eagerly=True`, the model runs in eager mode, allowing immediate evaluation of operations.

3.4 Key Takeaway

This represent the most important chapter of the work. Not only it describes in details a solution developed using ML but also provide a starting point for the chapter that will follow; essentials to conclude the thesis. Highlights discussed during this chapter are mainly technical. Below the key takeaway.

- Project Overview and Objectives
- Methodology Employed used for the development (mainly focousing on the ML side)
- Learning Tasks which MEDUSA ML model is responsible for:
 1. Multi-Label Hierarchical Classification: Labeling findings with multiple labels using the hierarchical structure both for contextual classification and better rappresentation.
 2. Similarity Detection: Using an embedding space of 350 dimensions, compute the similarity of findings.
 3. Severity Prediction: How MEDUSA is able to predicts severity as a task of regression.

- Custom Hugging Face Pipeline: How MEDUSA leverages the Hugging Face library for tokenization and preparing the model.
- A small Framework Comparative Analysis: Comparing the modes of execution between TensorFlow with PyTorch, MEDUSA mainly uses TensorFlow 2.x.

Chapter 4

Impact Assessment and Prospects

In this chapter, we evaluate the contribution of the MEDUSA system to the preparation of ARs and its further applications while trying to close the two previously formulated objectives of the thesis. The performance assessment and the gains attained should provide an understanding of the benefits and limitations of the tool. In addition, we explore more opportunities for applying MEDUSA and thus demonstrate its versatility and potential to bring improvements into many areas of financial supervision.

4.1 Evaluation Methodology and Performance Analysis



Figure 4.1: Before MEDUSA

In order to assess the effectiveness of MEDUSA, a multi-faceted evaluation methodology has to be employed. It had variables for metrics (Fig.4.2) that were quantitative and qualitative, so as to measure KPIs relevant to AR preparation, including accuracy, efficiency, consistency, and user satisfaction. This review had also considered the broader implications of the precision of the tool and its potential collaboration, which relates directly to the main objectives of the thesis: to review the influence of ML algorithms on ECB banking supervision, and problems related to interpretability and uncertainty.

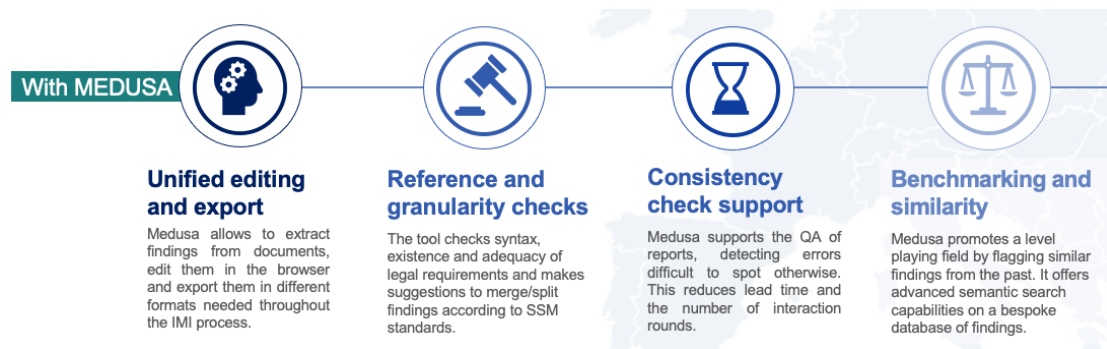


Figure 4.2: After MEDUSA

Performance was evaluated using several metrics. Accuracy could be measured by the classifications and suggestions outputted by MEDUSA against human supervisors' decisions and classifications. It would, therefore, assess the system's ability to replicate human decisions and classifications. Efficiency gains were assessed by the time taken to prepare ARs both with and without the use of MEDUSA. The measurement would bring out how much time was saved by automating tasks and reducing the amount of manual effort needed. Consistency was checked by looking at the different reports that dealt with similar findings. In this respect, it was checked the standardization of labels, the legal references, and the wording suggestions made by the system. Satisfaction from users was taken through surveys and feedback from supervisors using MEDUSA in the preparation of their report gave insight into how useful and easy to use this system was perceived to be. First, the baseline performance of human supervisors in preparing ARs without MEDUSA was measured. The MEDUSA models were trained on the historic AR dataset and fine-tuned for optimal classification and similarity detection tasks. Model parameters were adjusted to attain optimal performance on a separate validation set, ensuring good generalizability. Testing on the test set provided the final evaluation of the system and gave a measure of accuracy and effectiveness in real-life situations. After this, MEDUSA was offered for use by supervisors in their report preparation duties. Feedback obtained and efficiency gains noted during these trials were recorded and analyzed.

The system was found to be very accurate in classifying the issues correctly and providing the relevant legal references that very closely matched the decisions that human supervisors would have taken. Medusa has reduced the preparation time of ARs by a quantum, making the supervisors available for other complex analytical tasks. Standardization of labels and references brings the AR into a uniform format for consistency across various reports. The level of satisfaction expressed by the supervisors in using the system was very high as it was easy to use and reduced manual effort.



Figure 4.3: Assessment

As illustrated in Figure 4.3, various functionalities of MEDUSA were evaluated on a scale from “*Very Poor*” to “*Very Good*”. Extraction of findings from reports, ad hoc legal reference detection, and label classification all reached the best grades, thus performance for these features was good. Some functionalities, like export options or merge/split proposals for findings, unveiled potential for improvement.

The second indicator of performance is scalability. MEDUSA enabled further collaboration with other financial supervisory entities due to its development and deployment. For example, there is a proposed collaboration with the Banca d’Italia (BdI) and the search for synergies with their Findings Automation (FA) tool. The aim of the joint project, JEDAI (Joint Exploration for Developing Artificial Intelligence), was to understand what kind of synergies could be achieved between MEDUSA and FA, probably improving both systems, as considerations and datasets, like legal references, could be shared.

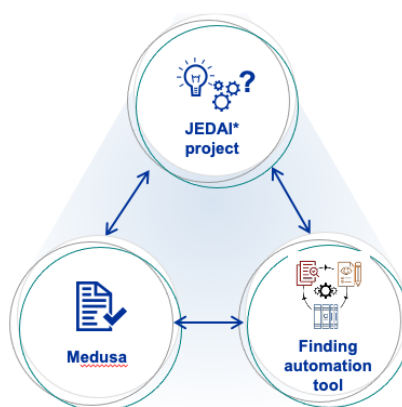


Figure 4.4: JEDAI Concept

The collaborative approach, summarized in Fig. 4.4 is also based on the SSM Digitalisation Blueprint (Fig. 4.5), which called for the exploitation of synergies between different SupTech projects in order to avoid duplication of efforts and to build stronger SSM-wide collaboration. By integrating MEDUSA with similar tools used by other entities, this developed solution will be able to be reused

in similar financial contexts, therefore increasing its impact and utility. These collaboration discussions have brought forward the organizational and technical challenges, like governance frameworks, but also structural differences in findings. Still, the potential for MEDUSA to be a foundational tool for broader supervisory applications is huge—very much in line with the thesis goal of assessing the impact of ML on ECB operations, addressing interpretability, and uncertainty.

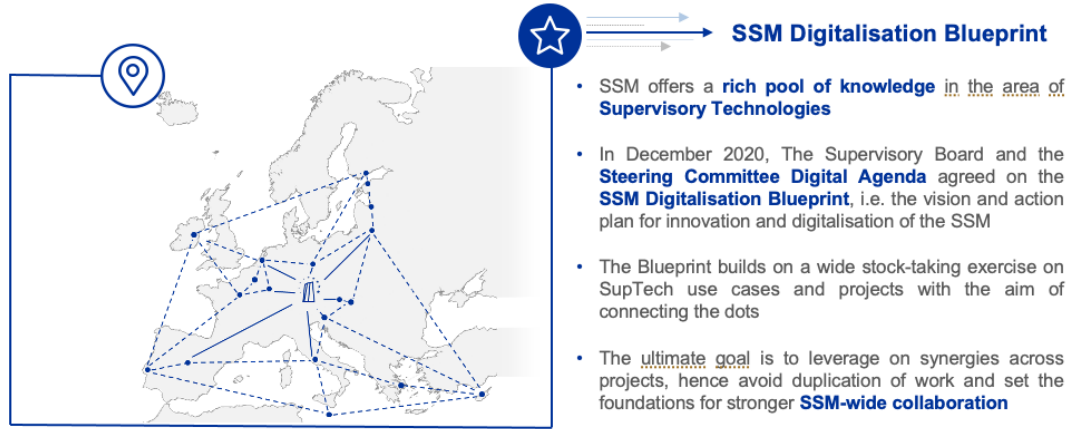


Figure 4.5: SSM Digitalisation Blueprint

4.2 Further possible Applications

As shown above, MEDUSA significantly enhances efficiency and consistency for AR preparation, thanks to the use of ML techniques. With a current implementation at great potential for further applications that can extend its utility and impact, this subsection shall explain them considering both goals: enhancing decision-making processes and ensuring interpretability and accountability in ML models.

MEDUSA, due to its design around the NLP core with hierarchical multi-label classification, is well suited for further evolution and the addition of more functionality. Some of its capabilities in these respects would be relevant to explore applications not just for streamlining operations but also for providing more profound insights with enhanced transparency and interpretability.

One of the possibilities to extend MEDUSA’s functionality is the use of LambdaMART—a very strong and fast algorithm for interpretable asset valuation. This aligns well with the second goals posed initially for the thesis: interpretability and uncertainty.

4.2.1 LambdaMART for Interpretable Valuation

LambdaMART is normally used in learning to rank. Learning to rank usually denotes that machine learning techniques are used to train a model with the view to automatically sort or prioritize items in a meaningful order. This would be very useful in scenarios where ranking lists of items against certain criteria needs to be

done. For example, LambdaMART can, in the context of MEDUSA, help sort out issues in ARs, making sure that the most critical ones are dealt with before others.

LambdaMART integrates gradient boosting with ranking and is hence in a position to cope with large datasets and complex interactions between features while remaining highly interpretable. Gradient Boosting works by creating an ensemble of decision trees where each subsequent tree corrects for errors of previous ones. This would let LambdaMART model complicated relationships in the data. A ranking component ensures that the model learns to rank items by priority in such a way that it maximizes value for a ranking-specific objective measure, like normalized Discounted Cumulative Gain (nDCG). This approach retains high levels of interpretability (Fig. 4.6) via: first, the use of decision trees as base learners provides an intuitive representation of how features interact to influence the final ranking. Being a partitioning-based tree, it is easy to trace back the path taken towards a specific decision. This structure lets the supervisor understand the reasoning behind each classification and decision, very important in retaining trust and accountability.

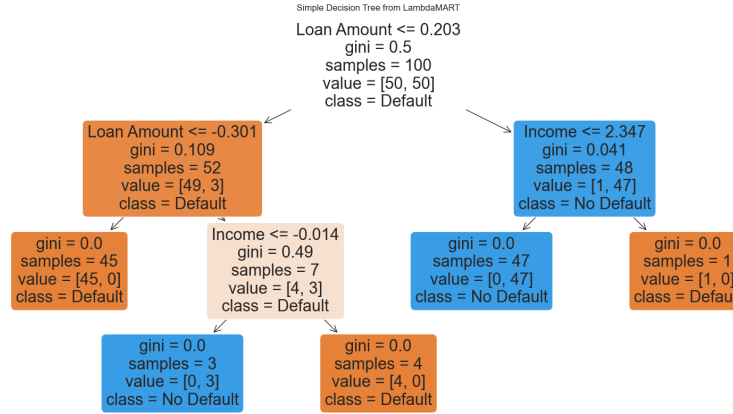


Figure 4.6: Example Decision Tree from LambdaMART showing how decisions are made

Second, LambdaMART features an importance technique for each feature in the model prediction. It measures the contribution of features to the model's accuracy, hence giving information on how each feature would improve the accuracy of the model, therefore highlighting the most important factors that drive the ranking process. This information can be plotted and interpreted by supervisors for insight into what really drives asset valuation.

Another critical challenge in applying machine learning to financial supervision is to guarantee that model decisions are interpretable. Still better than our Hierarchical Multi-label approach, LambdaMART structures feature interactions so as to let supervisors realize the reasoning behind each classification and decision. MEDUSA processes and analyzes long financial documents using LambdaMART for big datasets and complex feature spaces.

No metric is more important than nDCG can be as far as the ranking tasks are concerned. It handles the position of every relevant item in the ranking while giving a logarithmic discount factor that penalizes items the further down they appear. The formula for DCG is:

$$DCG_k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

where rel_i is the relevance score at position i .

The nDCG metric normalizes the DCG by comparing it to the ideal ranking:

$$nDCG_k = \frac{DCG_k}{IDCG_k}$$

where $IDCG_k$ is the DCG of the ideal ranking.

The technical integration of LambdaMART inside MEDUSA can be organized in several basic steps. We describe one possible solution here. Historical ARs and their findings create a comprehensive training dataset with all the relevant features to financial supervision, like legal references, issue severity, or past decisions. For such preprocessing steps, the Hugging Face pipeline is in charge of ensuring that the data are clean and structured appropriately for training. BERT, integrated by the Hugging Face pipeline, extricates rich semantic features from textual data. These features capture the meaning of a word and its context within the ARs, thus allowing for their nuanced representation. After that, these features are fed into LambdaMART for ranking tasks and enhance its potential to model complex relations within the data. The prepared dataset is used in training the algorithm LambdaMART later on. The training process involves tuning the hyper-parameters with early stopping to prevent overfitting. During this stage, the model will learn how to recognize relevant features and interactions impacting the valuations of assets.

First, it will constrain LambdaMART to have only one feature per tree, modeling only main effects. This step reduces the dimensionality of the features and identifies the most impactful among them. It identifies the key drivers of asset valuations by first looking at individual features. Then, LambdaMART will identify and model interactions between those features; that is, interactions between the selected features will further refine the model’s accuracy without losing its interpretability. This will ensure the capturing of complex dependencies between features and hence enhance the predictive power of the model. The trained model is validated using a separate dataset to ensure its accuracy and generalizability.

By using this structured approach to feature interactions, LambdaMART provides clear and interpretable insights into the factors influencing asset valuations for MEDUSA. This aligns with the need for transparent decision-making processes in financial supervision.

To illustrate, consider a scenario where a supervisor is preparing an AR for a bank’s loan portfolio. Manually, one would have to manually go through various reports to identify key issues. With LambdaMART integrated into MEDUSA, it can go through historical ARs automatically to extract relevant features using BERT, ranking them by order of severity and relevance to issues at hand. Say that it had concluded that high loan-to-value ratios and low credit scores make a loan more likely to default. It is on this that LambdaMART focuses, enabling the supervisor to have a quick view of what the major risk factors are and make

informed decisions. Interpretability in the system will let the supervisor trace back the line of reasoning for every decision and thus instill faith in the automated process.

4.3 Key Takeaway

In this chapter we discussed the final performances of MEDUSA. We highlighted the achievements of the objectives established in the introduction (with a focus in identifying solutions for interpretability and uncertainty), we also highlighted performances, collaborations, limits, and possible solutions such as LambdaMART for more interpretability. Key Takeaway discussed during this chapter are:

- Evaluation Methodology: possible collaborations making it a salable tool, utility and limits
- Performance Analysis: how we can assess the real usefulness in the ECB context.
- Further possible application: improvements to make ML decisions more clear, understandable and eventually efficient. Proposing a solution by applying LambdaMART.

Chapter 5

Conclusion

This final chapter of the thesis revolves back to some of the topics introduced in the foreword.

In this thesis, we explored what role machine learning could play in financial operations using BERT and a hierarchical multi-label approach in classification. The backgrounds aimed to enhance the drafting of ARs with the MEDUSA project, for more efficient and consistent reporting. MEDUSA was presented as a case study, considering that this would be a very practical application of machine learning in finance. Its main objective was the automation of the AR drafting process while assuring consistency and regulatory standard compliance. MEDUSA is powered by several NLP techniques and complex learning tasks.

We saw that there were several objectives for MEDUSA. It also provided an efficiency gain by automating the compilation of draft landscapes, thereby eliminating hundreds of man-hours of work by supervisors. MEDUSA has been aiming for other key objectives too: consistency—standardizing finding and issue treatment across reports and across supervisors—and reduced variability from interpretation. Yet another key element of the mission underlying MEDUSA was the enhancement of the quality of decisions.

It provided supervisors with full-fledged tools to enable more informed data-driven decisions. In this methodology, there were several key steps: the first of which is data collection and preprocessing. Information of relevance to the study was extracted from the different ARs, cleaned, and transformed into the appropriate format. Afterward, the machine learning model would be developed where the BERT model would be trained to understand and classify findings precisely. A multi-label hierarchical classification approach was used; that was the best technique to be applied to provide a good classification of findings into multiple labels while respecting dependencies and relationships. For simplicity, measures were derived both for finding similarities and also to calculate issue-severity forecasting.

The results of the MEDUSA project were major improvements in the drafting process for ARs. The accuracy of findings classification was improved by the BERT model, such that supervisors would be confident of relying upon those automatic suggestions. MEDUSA standardised most of the variability due to individual interpretation through language and structure standardization of ARs. Automation of routine tasks enabled supervisors to focus on other critical aspects of their job, thereby improving productivity.

The successful implementations also raised several ethical and practical concerns related to the application of machine learning in financial reporting. First was the issue of understanding and explaining decisions with the “black box” nature of machine learning models. Bias within the training data may mean that unfair or discriminatory outcomes would need to be very carefully monitored for and controlled. Inevitably, clear lines of responsibility for decisions made by the automated system had to be established, particularly in a regulatory context.

On the other side, the MEDUSA project is only the door opener for further improvements and applications. Realization and further explorations about strategies to gain efficiency in the MEDUSA pipeline and research about how to use LambdaMART for more interpretable asset valuation could turn up the transparency and understandability of the decisions made.

In general, this thesis has shown how machine learning in finance, and hierarchical multilabel classification using the model BERT in particular, could really change financial reporting at the European Central Bank. Even though results are promising, this project has stressed that facing both ethical and practical challenges is a condition to be able to put these technologies at the service effectively and responsibly in financial supervision.

Bibliography

- [1] Algorithmic bias detection and mitigation: Best practices and policies to reduce consumer harms, 2023. URL <https://www.brookings.edu/research/algorithmic-bias-detection-and-mitigation-best-practices-and-policies/>. Brookings Institution.
- [2] The ethics of ai addressing bias, privacy, and accountability in machine learning, 2024. URL <https://www.cloudthat.com/resources/the-ethics-of-ai-addressing-bias-privacy-and-accountability-in-machine-learning>. CloudThat.
- [3] Bringing artificial intelligence to banking supervision, 2024. URL <https://www.bankingsupervision.europa.eu/press/pr/date/2024/html/ssm.ml.en.html>. European Central Bank.
- [4] Ai ethics: Exploring the ethical dimensions of ai development, 2024. URL <https://www.meritdata-tech.com/articles/ai-ethics>. Merit Data Tech.
- [5] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, ..., and Xiaoqiang Zheng. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2016. Retrieved from <https://arxiv.org/abs/1603.04467>.
- [6] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, ..., and Xiaoqiang Zheng. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283. USENIX Association, 2016. Retrieved from <https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf>.
- [7] Rohan Agnihotri. Eager execution: An imperative interface to tensorflow, 2018. Retrieved from <https://www.tensorflow.org/guide/eager>.
- [8] Altair. Altair rapidminer, 2023. URL <https://altair.com/altair-rapidminer>.
- [9] Altoros. Tensorflow in finance: Discussing predictive analytics and budget planning, 2023. URL <https://www.altoros.com/blog/tensorflow-in-finance-discussing-predictive-analytics-and-budget-planning/>.
- [10] Michael R. Berthold, Nicolas Cebon, Fabian Dill, Thomas R. Gabriel, Tobias Kötter, Thomas Meinel, Peter Ohl, Christoph Sieb, Kilian Thiel, and Bernd Wiswedel. Knime: The konstanz information miner. In *Data Analysis, Machine Learning and Applications*, pages 319–326. Springer, Berlin, Heidelberg, 2009. Retrieved from https://link.springer.com/chapter/10.1007/978-3-540-78246-9_38.
- [11] Warren Buffett. Billionaire talks strategy with students columbia university group hears from famous alumnus berkshire hathaway. *Omaha World-Herald*, January 2 1994. Reported by Jim Rasmussen.
- [12] Dharaneish V C. Splitting a dataset into train, validation, and test sets using python. *Medium*, 2023. URL <https://medium.com/@dharaneishvc/>. Apr 26, 2023.
- [13] Confluent. Apache kafka: Benefits and use cases, 2023. URL <https://www.confluent.io/learn/apache-kafka-benefits-and-use-cases/>.

- [14] Datamation. Data mining use cases, 2023. URL <https://www.datamation.com/big-data/data-mining-use-cases/>.
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019. URL <https://arxiv.org/abs/1810.04805>.
- [16] Frank Elderson. Powers, ability and willingness to act – the mainstay of effective banking supervision. 2023. URL <https://www.ecb.europa.eu/press/key/date/2023/html/ecb.sp231207~a8b0fbe4e5.en.html>. European Central Bank.
- [17] Hugging Face. Bert: Pre-training of deep bidirectional transformers for language understanding, 2020. URL <https://huggingface.co/bert-base-uncased>. Accessed: 2024-09-12.
- [18] Bijit Ghosh. Empowering language models: Pre-training, fine-tuning, and in-context learning. *Medium*, 2023. URL <https://medium.com/@bijit211987/>. 6 min read, Jun 12, 2023.
- [19] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. URL <https://www.deeplearningbook.org/>.
- [20] Markus Hofmann and Ralf Klinkenberg. *RapidMiner: Data Mining Use Cases and Business Analytics Applications*. CRC Press, 2016. Retrieved from <https://www.taylorandfrancis.com/books/mono/10.1201/9781315399591>.
- [21] W. H. Inmon. *Building the Data Warehouse*. Wiley, 3rd edition, 2002.
- [22] Apache Kafka. Kafka: A distributed messaging system for log processing, 2011. Retrieved from <https://kafka.apache.org/documentation>.
- [23] R. Kimball and M. Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3rd edition, 2013.
- [24] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. URL [https://deepai.org/machine-learning-glossary-and-terms/adam-machine-learning#:~:text=Adam%20combines%20the%20advantages%20of%20two%20other%20extensions,Algorithm%20\(AdaGrad\)%20and%20Root%20Mean%20Square%20Propagation%20\(RMSProp\)](https://deepai.org/machine-learning-glossary-and-terms/adam-machine-learning#:~:text=Adam%20combines%20the%20advantages%20of%20two%20other%20extensions,Algorithm%20(AdaGrad)%20and%20Root%20Mean%20Square%20Propagation%20(RMSProp).).
- [25] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations (ICLR 2017)*, 2017. Retrieved from <https://arxiv.org/abs/1609.02907>.
- [26] KNIME. How day5 analytics centralized and automated 500+ financial processes, 2023. URL <https://www.knime.com/success-story/how-day5-analytics-centralized-and-automated-500-financial-processes>.
- [27] Claudio Lucchese. Data and web mining course. University course, 2023. Academic year 2023/2024, Università Ca’ Foscari of Venice.
- [28] Tom M. Mitchell. *Machine Learning*. McGraw Hill, New York, NY, 1997. URL <https://www.cs.cmu.edu/~tom/mlbook.html>.
- [29] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. MIT Press, Cambridge, MA, 2nd edition, 2018. URL <https://mitpress.mit.edu/books/foundations-machine-learning-second-edition>.
- [30] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, ..., and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. Retrieved from <https://arxiv.org/abs/1912.01703>.

- [31] Alec Radford, Jeffrey Wu, Dario Amodei, Jack Clark, et al. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019. URL <https://openai.com/research/better-language-models>.
- [32] Google Research. Bert: Bidirectional encoder representations from transformers. <https://research.google/pubs/pub47736/>, 2018. Accessed: 2024-09-12.
- [33] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, May 2019. doi: 10.1038/s42256-019-0048-x.
- [34] Mckinsey & Company Financial Services. Capturing the full value of generative ai in banking, 2023. URL https://www.mckinsey.com/industries/financial-services/our-insights/capturing-the-full-value-of-generative-ai-in-banking#.
- [35] M. Stonebraker and J. M. Hellerstein. Content-based retrieval systems. *ACM Computing Surveys*, 33(2):288–325, 2001.
- [36] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. Retrieved from <http://incompleteideas.net/book/the-book-2nd.html>.
- [37] Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, and Vipin Kumar. *Introduction to Data Mining*. Pearson, 2nd edition, 2018. ISBN 9780133128901. URL <https://www.pearson.com/en-us/subject-catalog/p/Tan-Introduction-to-Data-Mining-Subscription-2nd-Edition/P200000003204/9780134080284>.
- [38] Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, and Vipin Kumar. *Introduction to Data Mining*. Pearson, 2nd edition, 2018. ISBN 9780134080284. URL <https://archive.org/details/introductiontoda0000tanp>.
- [39] Ultralytics. Gpt: Generative pre-trained transformer. <https://www.ultralytics.com/glossary/gpt-generative-pre-trained-transformer>, 2023. Accessed: 2024-09-12.
- [40] R. Y. Wang and D. M. Strong. Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4):5–33, 1996.
- [41] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, et al. Transformers: State-of-the-art natural language processing, 2020. URL <https://arxiv.org/abs/1910.03771>. Accessed: 2024-09-12.
- [42] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alexander Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489. ACL, 2016. Retrieved from <https://www.aclweb.org/anthology/N16-1174/>.
- [43] Jiayu Zhou and Lantao Wei. Hierarchical classification with reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 365–372. AAAI Press, 2018. Retrieved from <https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16888>.